Targeted LLM Knowledge Injection

Agenda

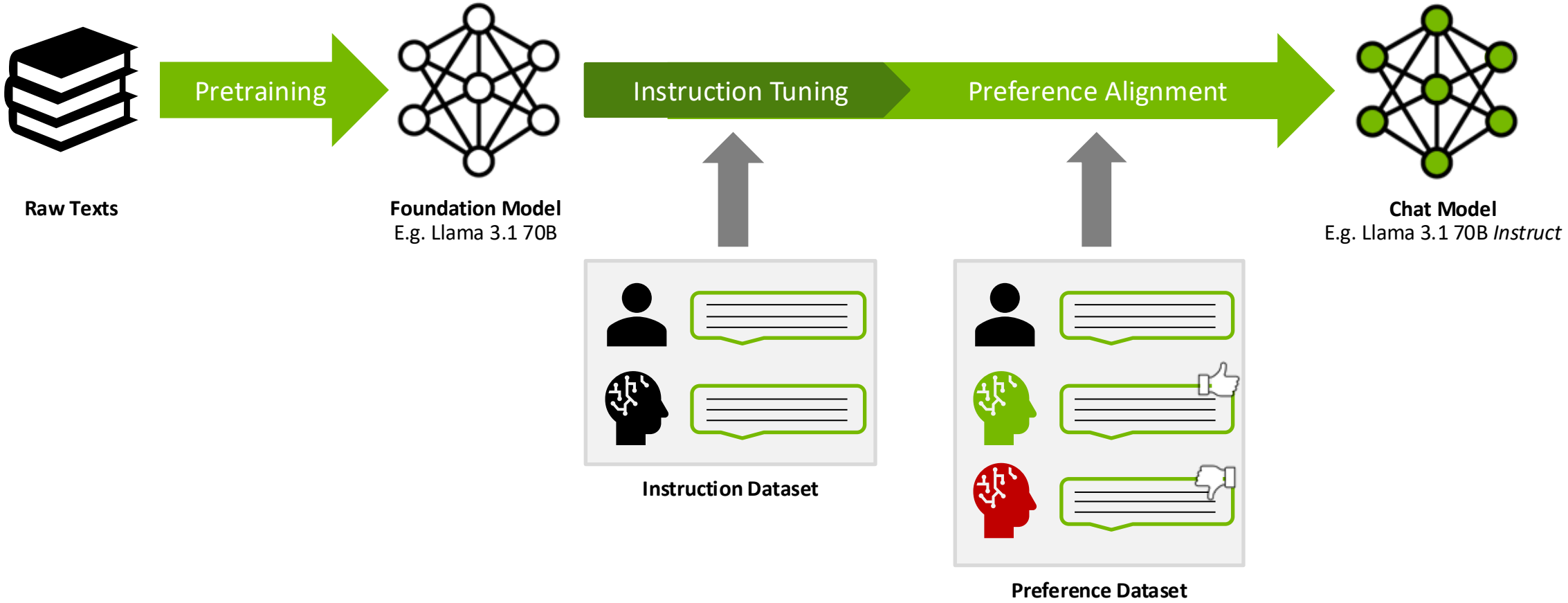
- **LLM Training Recap**
- Continued Pretraining - CPT
- Supervised Fine-Tuning - SFT
- Preference Alignment and RL
- Summary

LLM training recap

Pretrained first, aligned for conversation later

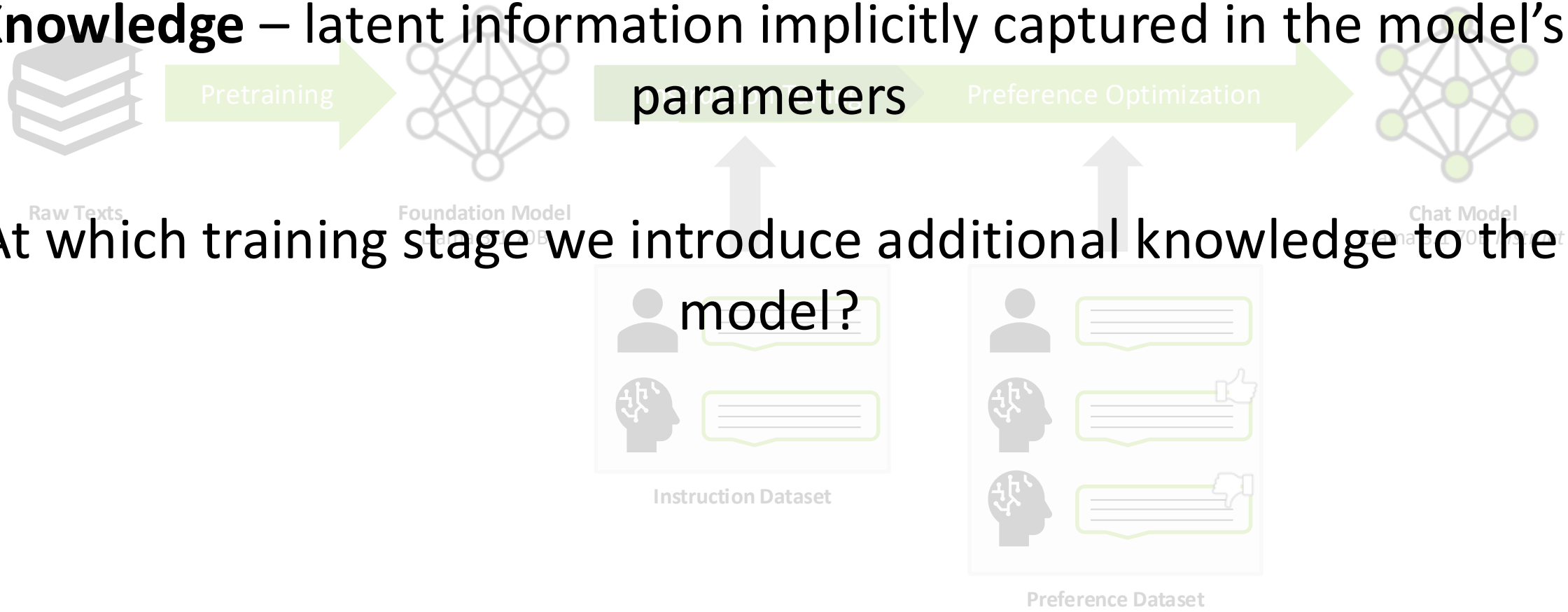
LLM training recap

Pretrained first, aligned for conversation via instruction and preference data



Knowledge in LLMs

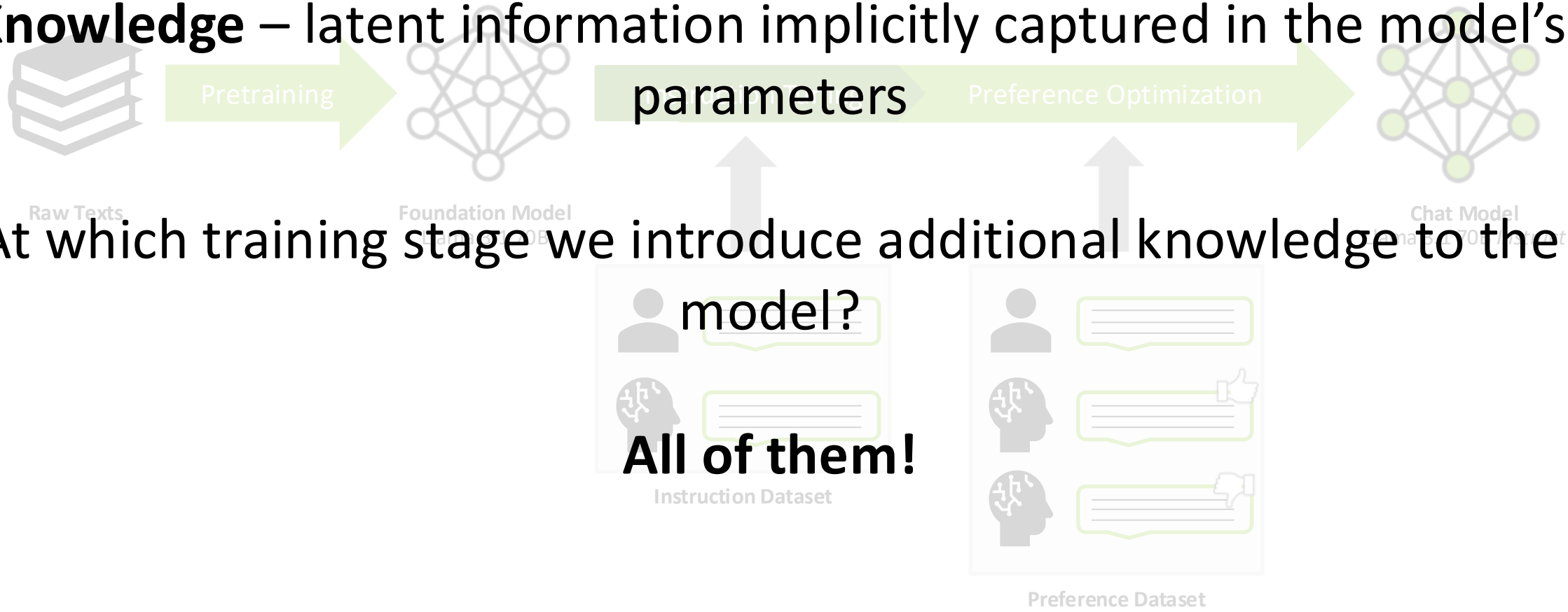
Knowledge – latent information implicitly captured in the model's parameters



At which training stage we introduce additional knowledge to the model?

Knowledge in LLMs

Knowledge – latent information implicitly captured in the model's parameters

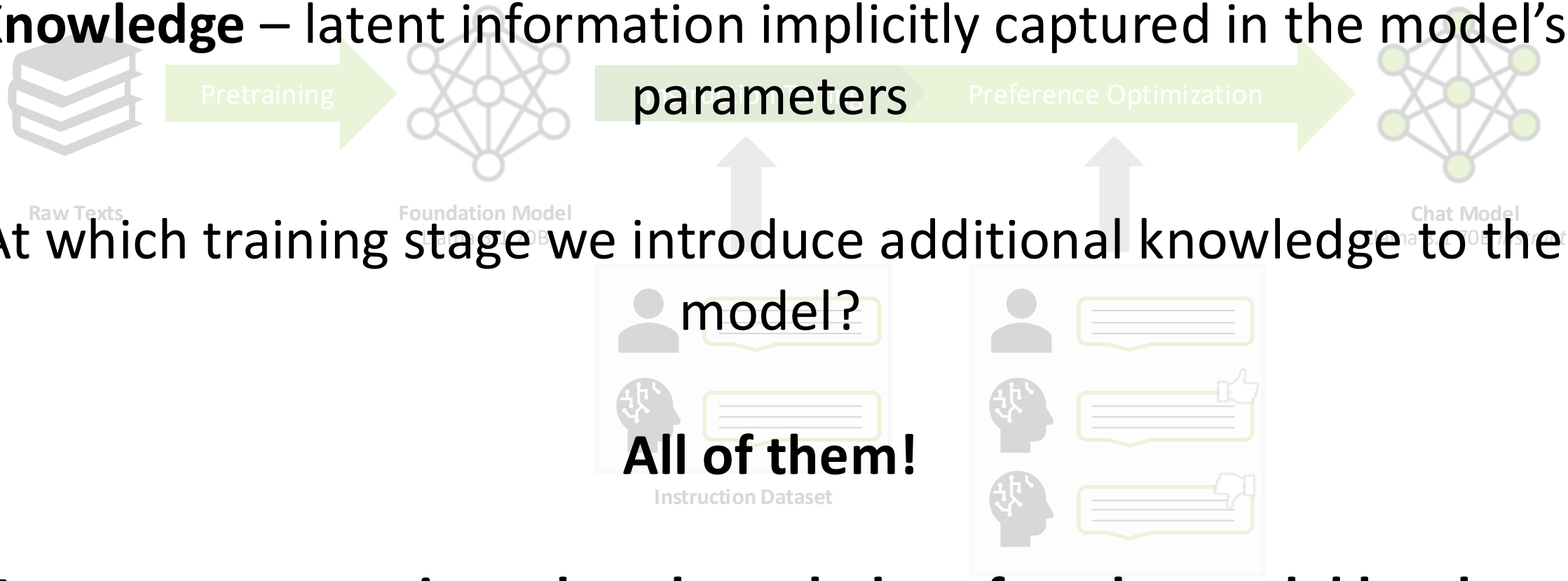


At which training stage we introduce additional knowledge to the model?

All of them!

Knowledge in LLMs

Knowledge – latent information implicitly captured in the model's parameters



At which training stage we introduce additional knowledge to the model?

All of them!

Moreover, we can introduce knowledge after the model has been trained with the same techniques!

Why do we need to introduce new knowledge?

Model knows only what it saw before the dataset cutoff date

Who is the current Pope?

The current Pope of the Catholic Church is **Pope Francis.**

Up-to-Date Knowledge

Why do we need to introduce new knowledge?

Model has limited knowledge in specialized domains

Who is the current Pope?

The current Pope of the Catholic Church is **Pope Francis.**

Up-to-Date Knowledge

Do JSE-listed dividends held in a Swiss trust trigger CRS reporting for a Japanese settlor?

As a large language model, I honestly have no idea and will hallucinate something instead.

Domain-Specific Knowledge

Why do we need to introduce new knowledge?

Model requires additional training for complex skills

Who is the current Pope?

The current Pope of the Catholic Church is **Pope Francis.**

Up-to-Date Knowledge

Do JSE-listed dividends held in a Swiss trust trigger CRS reporting for a Japanese settlor?

As a large language model, I honestly have no idea and will hallucinate something instead.

Domain-Specific Knowledge

Can you query our internal database for me?

I'm sorry, I haven't been trained to do so (yet!).

Additional Skills

Why do we need to introduce new knowledge?

Model was trained on English-centric data and lacks cultural knowledge

Who is the current Pope?

The current Pope of the Catholic Church is **Pope Francis.**

Up-to-Date Knowledge

Do JSE-listed dividends held in a Swiss trust trigger CRS reporting for a Japanese settlor?

As a large language model, I honestly have no idea and will hallucinate something instead.

Domain-Specific Knowledge

Can you query our internal database for me?

I'm sorry, I haven't been trained to do so (yet!).

Additional Skills

In Japan, is it appropriate to hand a business card with one hand during a first meeting?

Yes, certainly, that's perfectly fine.

Cultural or Regional Adaptation

Agenda

- LLM Training Recap
- **Continued Pretraining - CPT**
- Supervised Fine-Tuning - SFT
- Preference Alignment and RL
- Summary

LLM pretraining = self-supervised learning

Labels are not needed if we predict the next token!

A few thousand previous tokens for context

Predict the next token or group of tokens

Through hard work, he supported himself and his •••

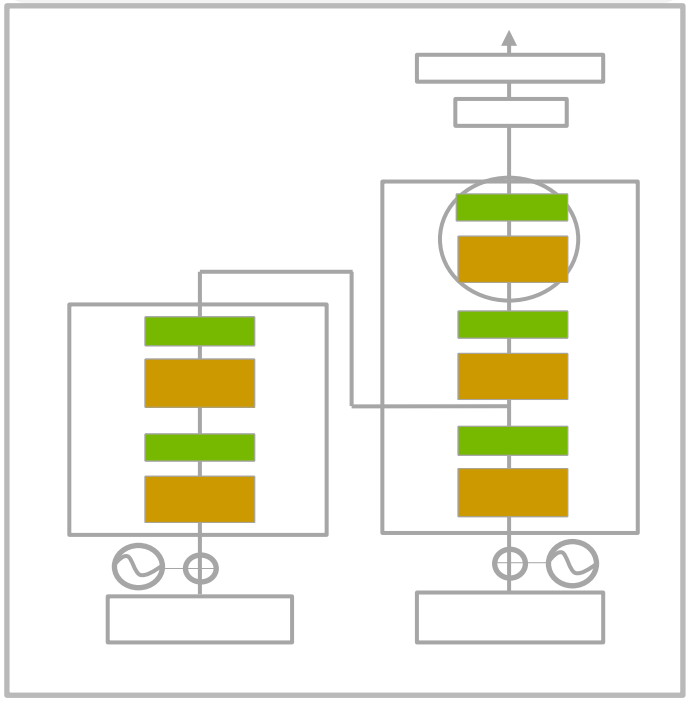
Because it crossed state lines, that criminal behavior attracted the attention of the •••

Bender Rodríguez was the one of the stars of the television show •••

```
// loop over the string
int i;
for (i = 0; i < •••
```

This restaurant was fabulous! My star rating is •••

Transformer Architecture
Deep Neural Network



"family"

"*FBI*"

"Futurama"

"strlen"

"five"

Continued pretraining = domain-specific self-supervised learning

Teaching the model domain-specific knowledge

A few thousand previous tokens for context

Predict the next token or group of tokens

Co-administration of CYP3A4 inhibitors like •••

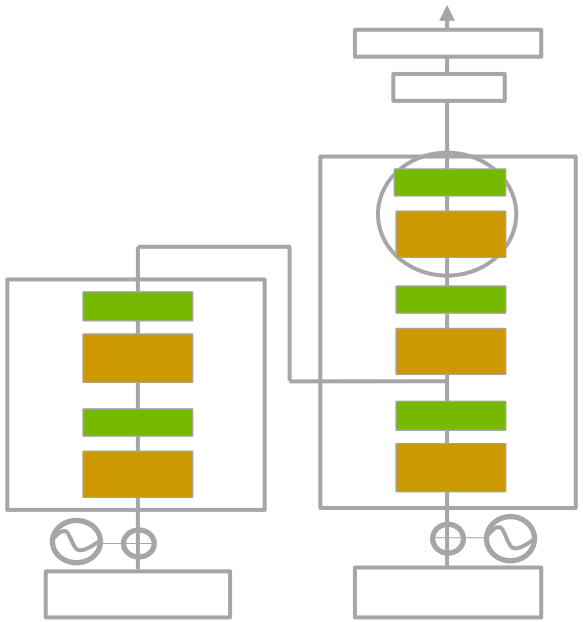
Clopidogrel's antiplatelet activity is attenuated in •••

Rifampin induces both CYP450 enzymes and •••

The QT-prolonging effect of macrolides like •••

Biologics targeting IL-17A, such as •••

Transformer Architecture
Deep Neural Network



"ketoconazole"

"CYP2C19"

"P-glycoprotein"

"erythromycin"

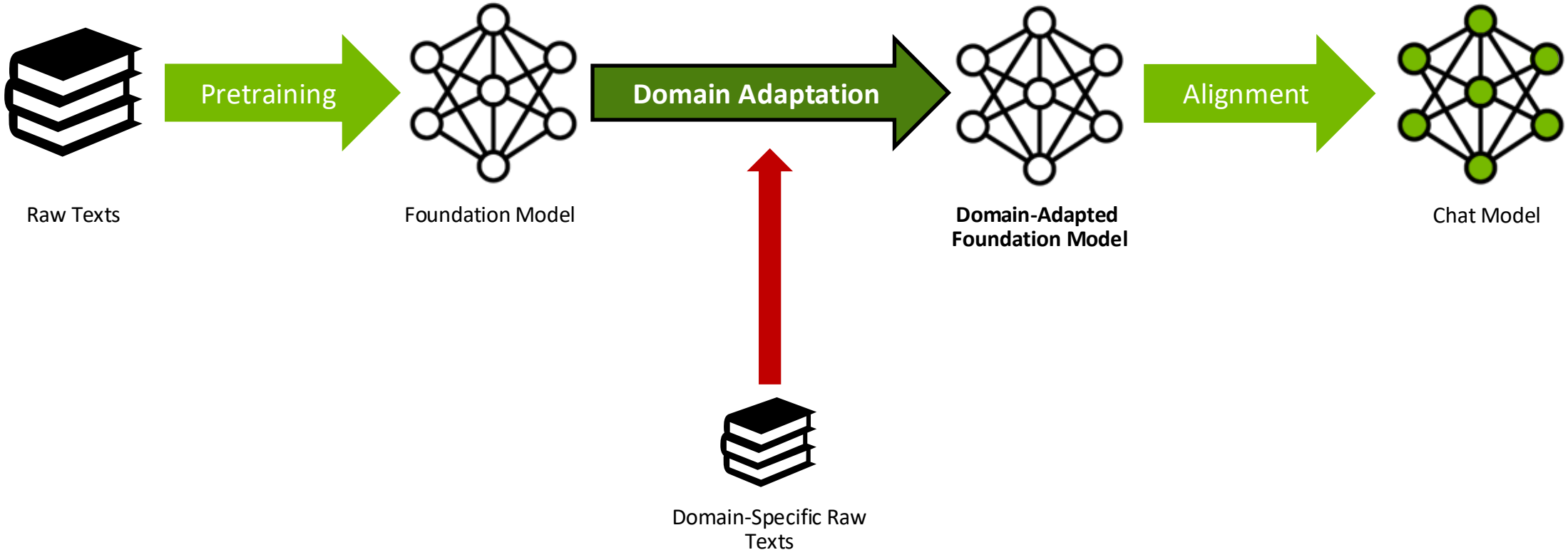
"secukinumab"

Back to LLM training

When to conduct continued pretraining?

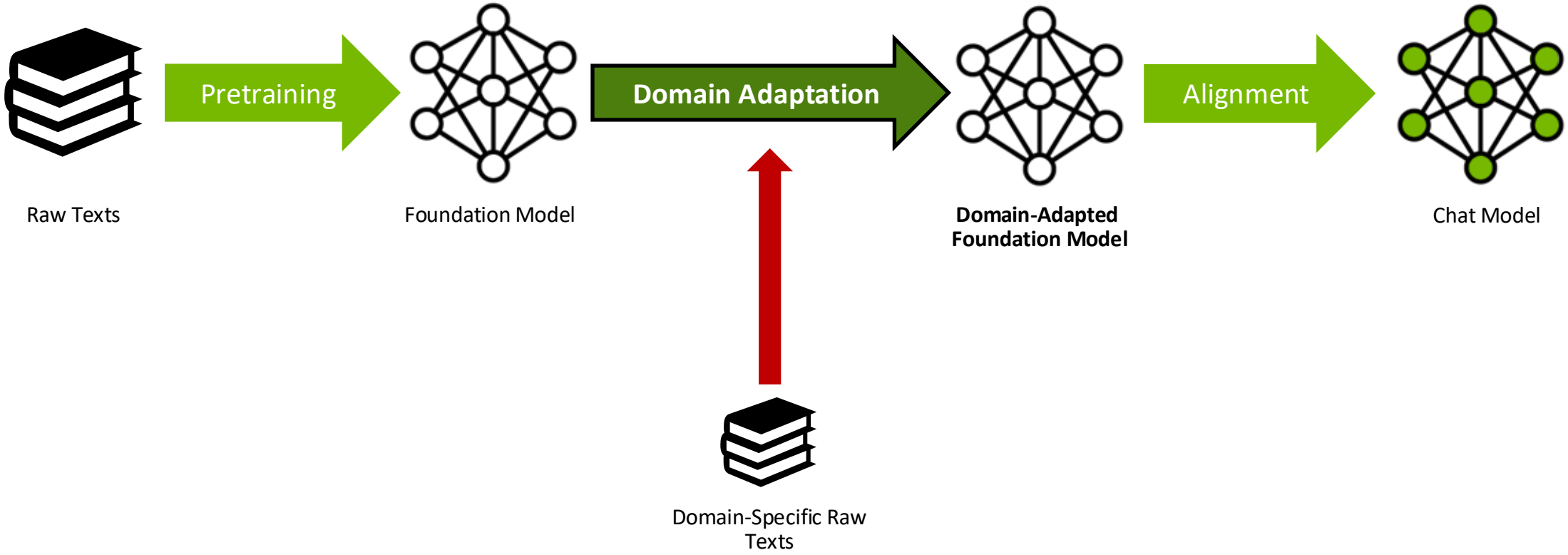
Continued pretraining follows original pretraining

LLM is still trained on next token prediction task, not on instruction data



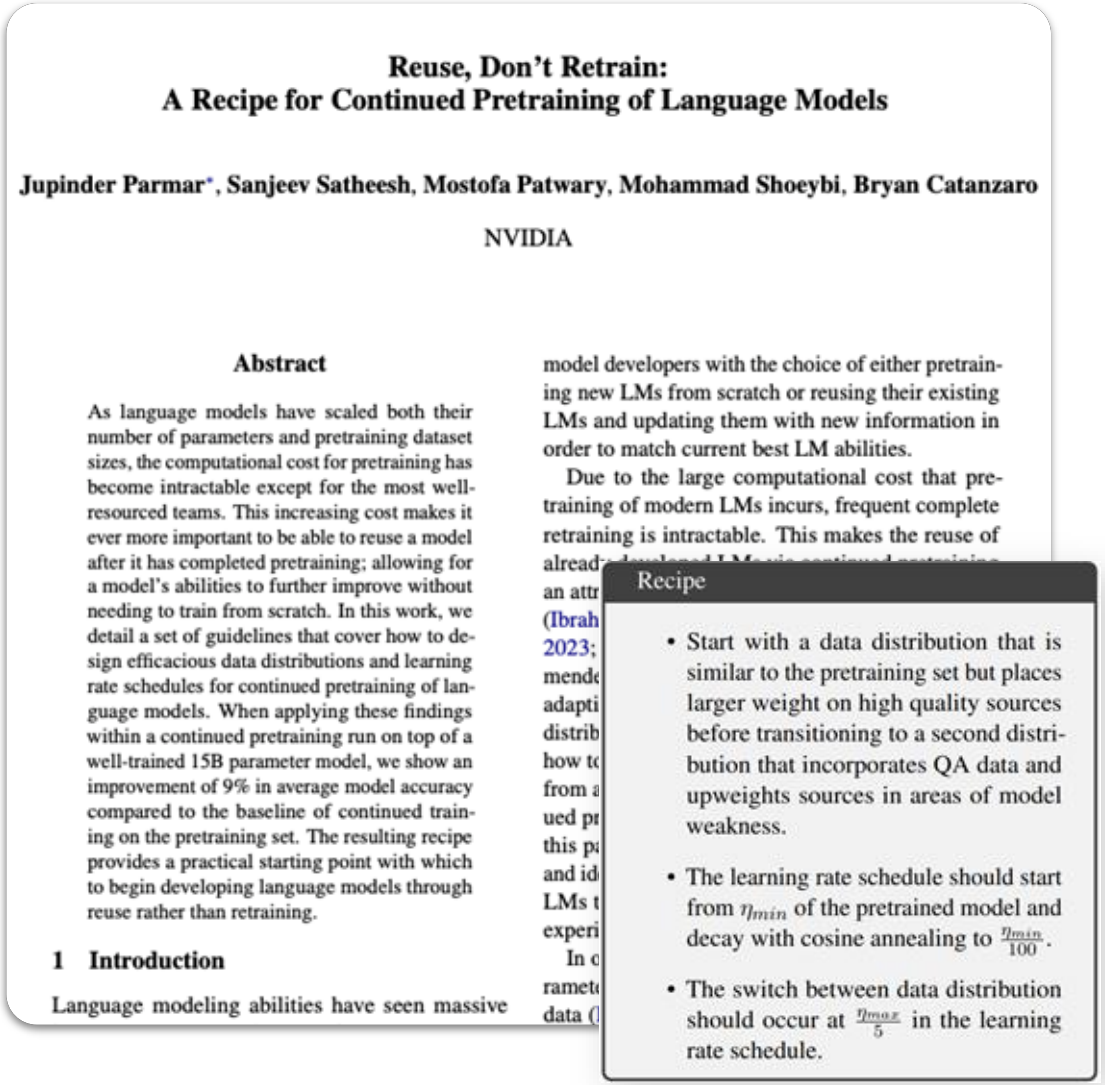
Continued pretraining follows original pretraining

LLM is still trained on next token prediction task, not on instruction data



Continued pretraining generally requires repeating alignment

Continued pretraining in practice



Reuse, Don't Retrain:
A Recipe for Continued Pretraining of Language Models

Jupinder Parmar*, **Sanjeev Satheesh**, **Mostofa Patwary**, **Mohammad Shoeybi**, **Bryan Catanzaro**

NVIDIA

Abstract

As language models have scaled both their number of parameters and pretraining dataset sizes, the computational cost for pretraining has become intractable except for the most well-resourced teams. This increasing cost makes it ever more important to be able to reuse a model after it has completed pretraining; allowing for a model's abilities to further improve without needing to train from scratch. In this work, we detail a set of guidelines that cover how to design efficacious data distributions and learning rate schedules for continued pretraining of language models. When applying these findings within a continued pretraining run on top of a well-trained 15B parameter model, we show an improvement of 9% in average model accuracy compared to the baseline of continued training on the pretraining set. The resulting recipe provides a practical starting point with which to begin developing language models through reuse rather than retraining.

1 Introduction

Language modeling abilities have seen massive

model developers with the choice of either pretraining new LMs from scratch or reusing their existing LMs and updating them with new information in order to match current best LM abilities.

Due to the large computational cost that pretraining of modern LMs incurs, frequent complete retraining is intractable. This makes the reuse of

Recipe

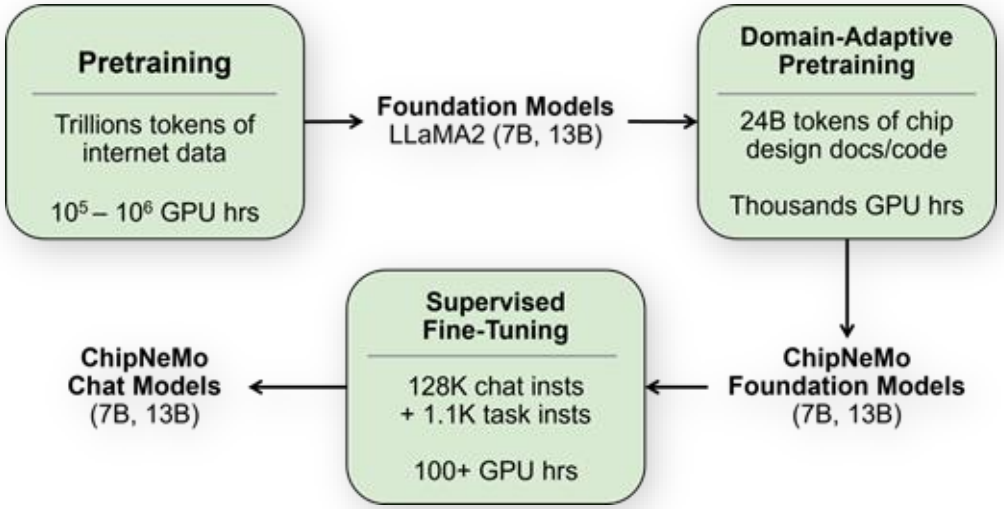
- Start with a data distribution that is similar to the pretraining set but places larger weight on high quality sources before transitioning to a second distribution that incorporates QA data and upweights sources in areas of model weakness.
- The learning rate schedule should start from η_{min} of the pretrained model and decay with cosine annealing to $\frac{\eta_{min}}{100}$.
- The switch between data distribution should occur at $\frac{\eta_{max}}{5}$ in the learning rate schedule.

Takeaways:

- **Data:**
  - Blend data from original pretraining set with new data
  - Increase weight on high-quality data sources in original data
  - Upweight categories on which model underperforms (STEM, code, …)
  - Including Q&A-style examples in new data improves accuracy
- **Training Recipe:**
  - Start with final learning rate, decay to 1% . Use same scheduler, do not use LR warmup
  - Loading optimizer states has low impact on performance – can use open-source LLMs

Parmar, Jupinder, et al. "Reuse, Don't Retrain: A Recipe for Continued Pretraining of Language Models." *arXiv preprint arXiv:2407.07263* (2024).

Continued pretraining in practice



Takeaways:

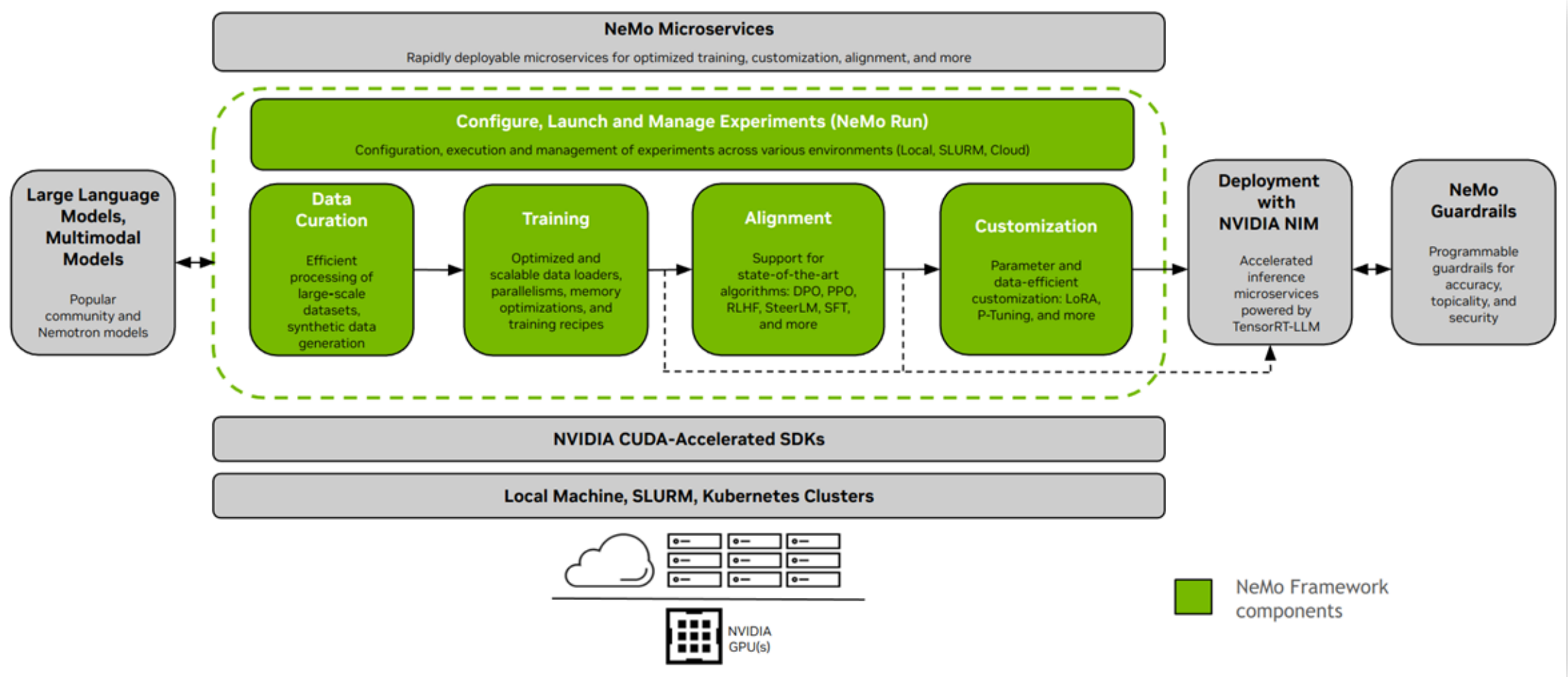
- Domain Adaptation (DAPT) was effective in adapting a LLM to chip design domain
- DAPT leads to performance improvement at a fraction of pretraining cost (≈1.5%)
- Tokenizer domain adaptation leads to higher tokenization and training efficiency
- Full-parameter DAPT outperforms PEFT on domain tasks
- Including a small percentage of public data helps maintain language capabilities

- **Model-only ChipNeMo performance is comparable to a usual chat model + RAG**

ChipNeMo: Domain-Adapted LLMs for Chip Design. Mingjie Liu et al. arXiv:2311.00176.

Nemo Framework

A suite of tools for large language models and multimodal models

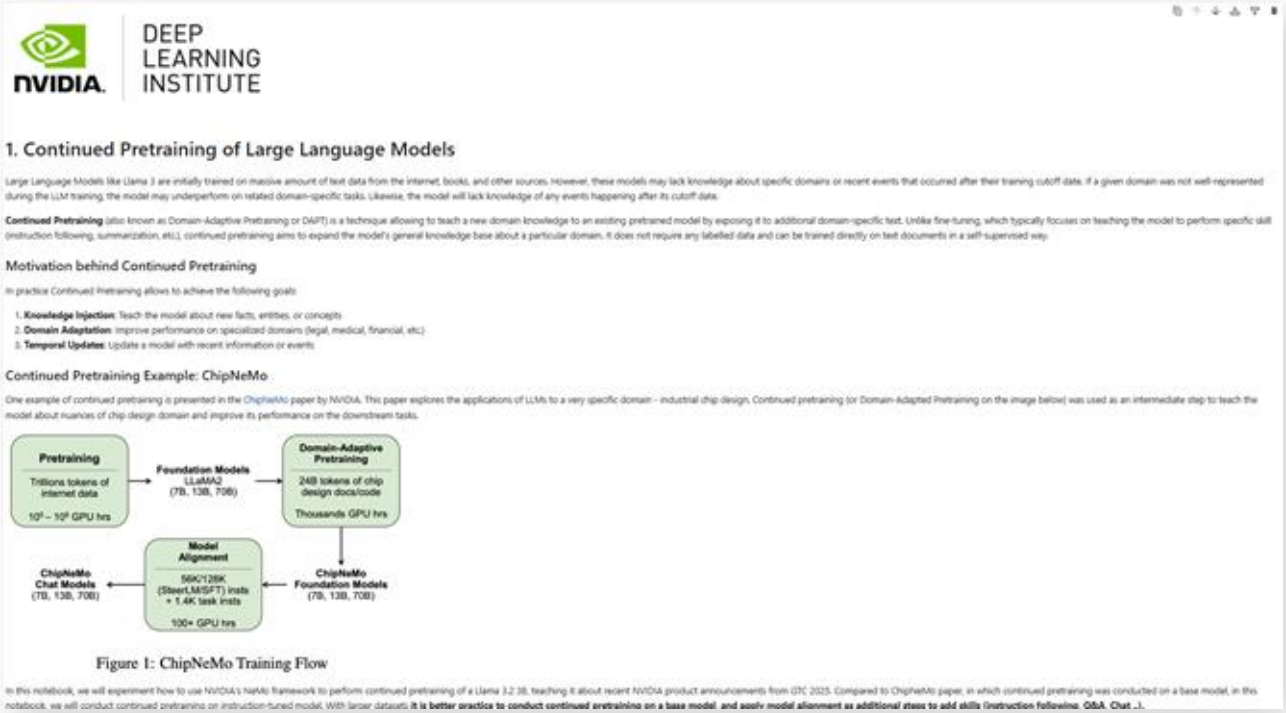


Overview — NVIDIA NeMo Framework User Guide

The lab

Notebook 1: Continued pretraining of large language models

1. Evaluate the Llama-3.2-3B-Instruct
2. Prepare the dataset
3. Initialize the continued pretraining recipe
4. Run continued pretraining
5. Evaluate the tuned model - did we learn anything?
6. Continued pretraining vs. RAG



NVIDIA DEEP LEARNING INSTITUTE

1. Continued Pretraining of Large Language Models

Large Language Models like Llama 3 are initially trained on massive amount of text data from the internet, books, and other sources. However, these models may lack knowledge about specific domains or recent events that occurred after their training cutoff date. If a given domain was not well-represented during the LLM training, the model may underperform on related domain-specific tasks. Likewise, the model will lack knowledge of any events happening after its cutoff date.

Continued Pretraining (also known as Domain-Adaptive Pretraining or DAPT) is a technique allowing to teach a new domain knowledge to an existing pretrained model by exposing it to additional domain-specific text. Unlike fine-tuning, which typically focuses on teaching the model to perform specific skill (instruction following, summarization, etc.), continued pretraining aims to expand the model's general knowledge base about a particular domain. It does not require any labelled data and can be trained directly on text documents in a self-supervised way.

Motivation behind Continued Pretraining

In practice Continued Pretraining allows to achieve the following goals:

1. **Knowledge Injection**: Teach the model about new facts, entities, or concepts
2. **Domain Adaptation**: Improve performance on specialized domains (legal, medical, financial, etc)
3. **Temporal Updates**: Update a model with recent information or events

Continued Pretraining Example: ChipNeMo

One example of continued pretraining is presented in the ChipNeMo paper by NVIDIA. This paper explores the applications of LLMs to a very specific domain - industrial chip design. Continued pretraining (or Domain-Adapted Pretraining on the image below) was used as an intermediate step to teach the model about nuances of chip design domain and improve its performance on the downstream tasks.

Figure 1: ChipNeMo Training Flow

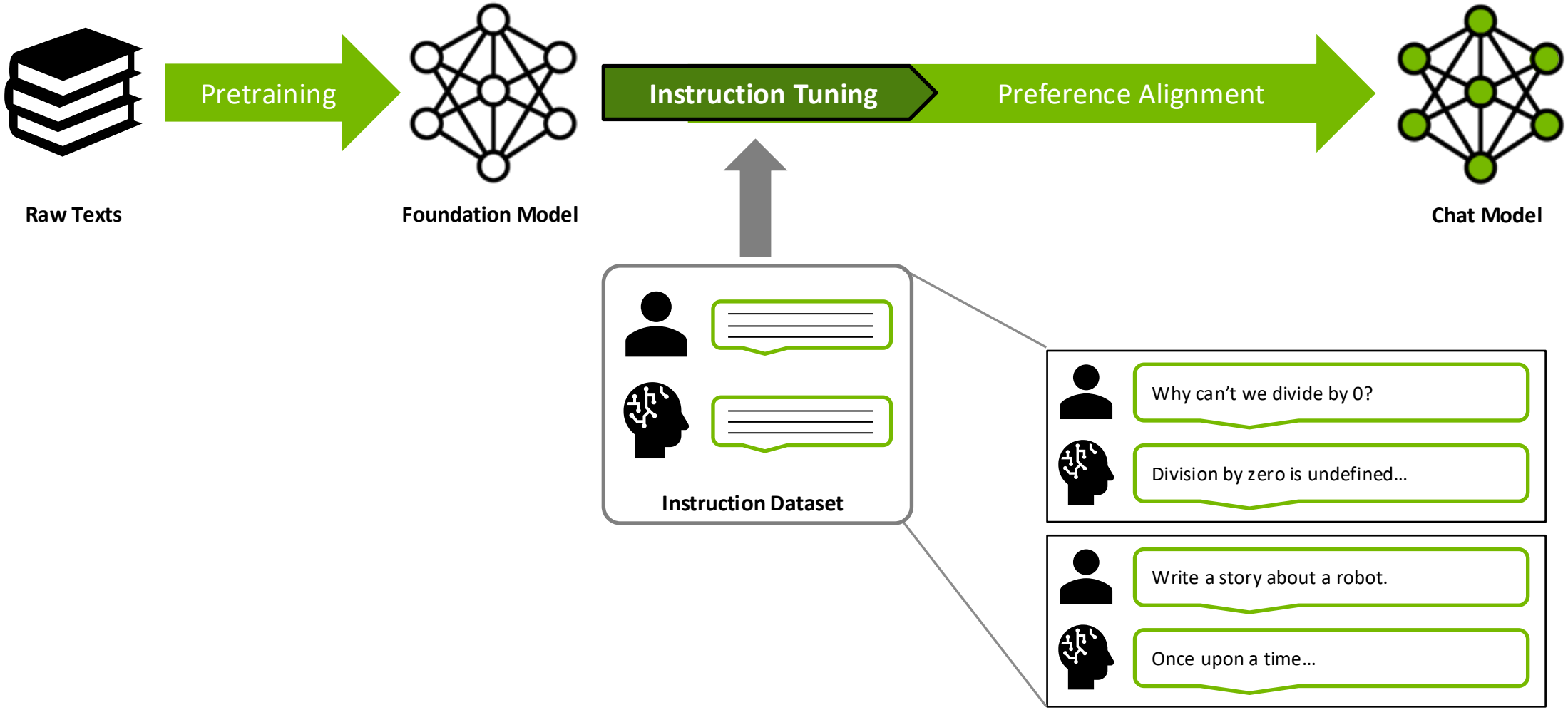
In this notebook, we will experiment how to use NVIDIA's NeMo framework to perform continued pretraining of a Llama 3.2 3B, teaching it about recent NVIDIA product announcements from GTC 2025. Compared to ChipNeMo paper, in which continued pretraining was conducted on a base model, in this notebook, we will conduct continued pretraining on instruction-tuned model. With larger datasets **it is better practice to conduct continued pretraining on a base model, and apply model alignment as additional steps to add skills (instruction following, Q&A, Chat ...).**

Agenda

- LLM Training Recap
- Continued Pretraining - CPT
- **Supervised Fine-Tuning - SFT**
- Preference Alignment and RL
- Summary

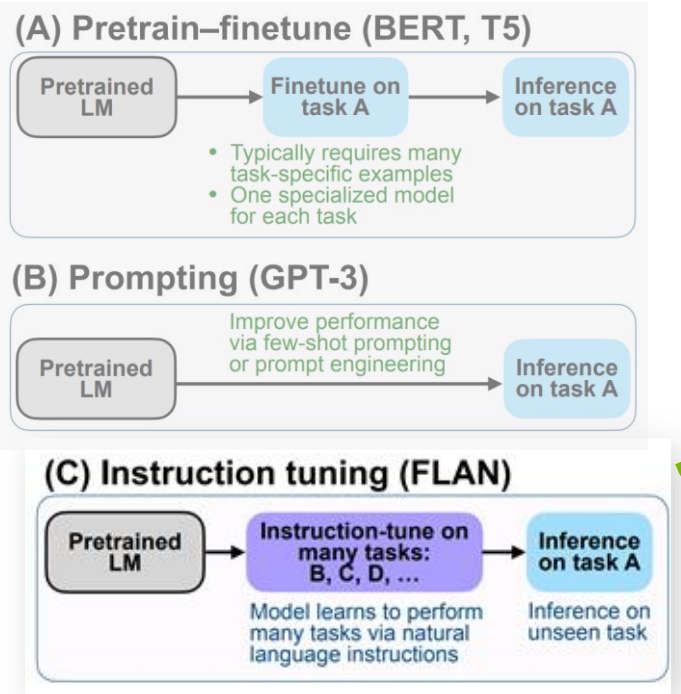
Supervised fine-tuning follows pretraining

Training on labeled conversation/task-specific examples



First step towards chat LLMs

Google Research's FLAN



Finetune on many tasks ("instruction-tuning")

Input (Commonsense Reasoning)

Here is a goal: Get a cool sleep on summer days.

How would you accomplish this goal?

OPTIONS:

-Keep stack of pillow cases in fridge.

-Keep stack of pillow cases in oven.

Target

keep stack of pillow cases in fridge

Input (Translation)

Translate this sentence to Spanish:

The new office building was built in less than three months.

Target

El nuevo edificio de oficinas se construyó en tres meses.

Sentiment analysis tasks

Coreference resolution tasks

...

Inference on unseen task type

Input (Natural Language Inference)

Premise: At my age you will probably have learnt one lesson.

Hypothesis: It's not certain how many lessons you'll learn by your thirties.

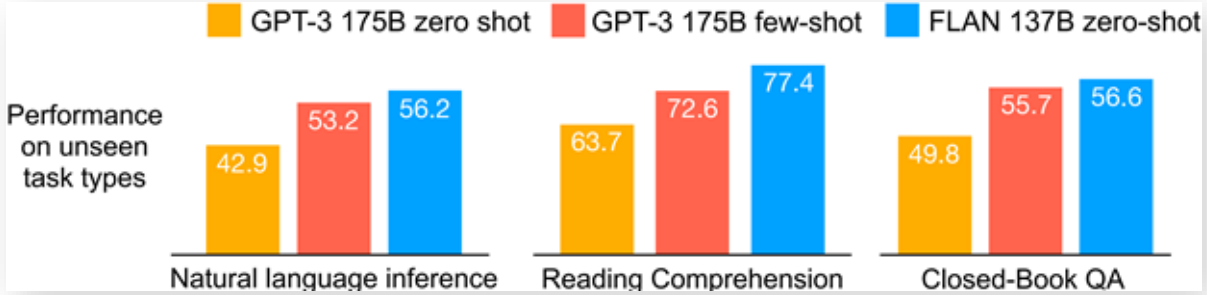
Does the premise entail the hypothesis?

OPTIONS:

-yes -it is not possible to tell -no

FLAN Response

It is not possible to tell



Wei, Jason, et al. "Finetuned language models are zero-shot learners." *arXiv preprint arXiv:2109.01652* (2021).

The origins of ChatGPT

From GPT-3 to InstructGPT

Training language models to follow instructions with human feedback

Long Ouyang* Jeff Wu* Xu Jiang* Diogo Almeida* Carroll L. Wainwright*
Pamela Mishkin* Chong Zhang Sandhini Agarwal Katarina Slama Alex Ray
John Schulman Jacob Hilton Fraser Kelton Luke Miller Maddie Simens
Amanda Askell† Peter Welinder Paul Christiano*†
Jan Leike* Ryan Lowe*
OpenAI

Abstract

Making language models bigger does not inherently make them better at following a user's intent. For example, large language models can generate outputs that are untruthful, toxic, or simply not helpful to the user. In other words, these models are not *aligned* with their users. In this paper, we show an avenue for aligning language models with user intent on a wide range of tasks by fine-tuning with human feedback. Starting with a set of labeler-written prompts and prompts submitted through the OpenAI API, we collect a dataset of labeler demonstrations of the desired model behavior, which we use to fine-tune GPT-3 using supervised learning. We then collect a dataset of rankings of model outputs, which we use to further fine-tune this supervised model using reinforcement learning from human feedback. We call the resulting models *InstructGPT*. In human evaluations on our prompt distribution, outputs from the 1.3B parameter InstructGPT model are preferred to outputs from the 175B GPT-3, despite having 100x fewer parameters. Moreover, InstructGPT models show improvements in truthfulness and reductions in toxic output generation while having minimal performance regressions on public NLP datasets. Even though InstructGPT still makes simple mistakes, our results show that fine-tuning with human feedback is a promising direction for aligning language models with human intent.

Takeaways:

- **First example of model alignment for Chat**
- Three-step process involving human feedback:
  1. **Supervised Fine-Tuning**
  2. Reward Model Training (*more on it later*)
  3. Reinforcement Learning with PPO (*more on it later*)
- 1.3B Instruct GPT outperforms 175B GPT-3
  - Smaller model can be made more effective on specific tasks or domains
- Model alignment represents preferences of labelers and researchers
  - Open-source LLMs do not represent all users

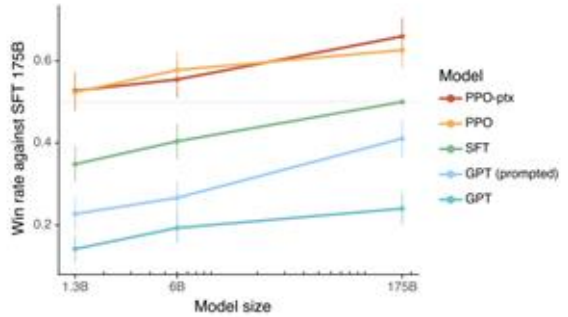


Figure 1: Human evaluations of various models on our API prompt distribution, evaluated by how often outputs from each model were preferred to those from the 175B SFT model. Our InstructGPT models (PPO-ptx) as well as its variant trained without pretraining mix (PPO) significantly outperform the GPT-3 baselines (GPT, GPT prompted); outputs from our 1.3B PPO-ptx model are preferred to those from the 175B GPT-3. Error bars throughout the paper are 95% confidence intervals.

Ouyang, Long, et al. "Training language models to follow instructions with human feedback." Advances in neural information processing systems 35 (2022): 27730-27744

First steps towards LLM reasoning

Chain-of-thought prompting teaches models to “think”

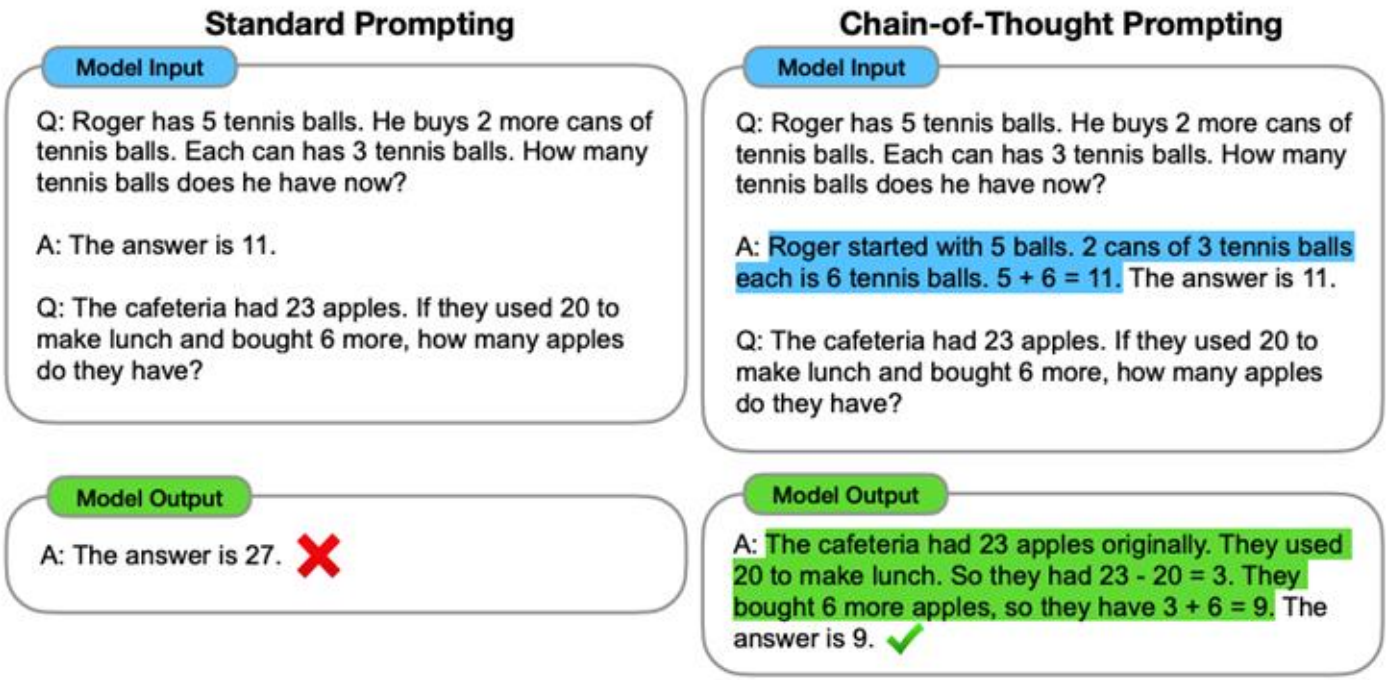


Figure 1: Chain-of-thought prompting enables large language models to tackle complex arithmetic, commonsense, and symbolic reasoning tasks. Chain-of-thought reasoning processes are highlighted.

Wei, Jason, et al. "Chain-of-thought prompting elicits reasoning in large language models." Advances in neural information processing systems 35 (2022): 24824-24837.

Reasoning with LLMs

Example: "Write an efficient Python function to check if the number is both prime and divisible by 5"

Reasoning **OFF**

Correct by not efficient/best answer

```python
def is_prime(n):
    """Check if a number is prime."""
    if n <= 1:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False
    max_divisor = int(n**0.5) + 1 # Correct but O(sqrt(N))
    for d in range(3, max_divisor, 2):
        if n % d == 0:
            return False
    return True

def is_divisible_by_5(n):
    """Check if a number is divisible by 5."""
    return n % 5 == 0

def is_prime_and_divisible_by_5(n):
    """Check if a number is both prime and divisible by 5."""
    return is_prime(n) and is_divisible_by_5(n)
```

Reasoning **ON**

Final answer is correct and efficient. But takes tokens to generate

```
<think>
Okay, I need to write a Python function that checks if a number is both prime and
divisible by 5. Hmm, let's think about the requirements here.

First, the number has to be prime. A prime number is a number greater than 1 that has no
divisors other than 1 and itself. Then, it also needs to be divisible by 5. So, the
number must be divisible by 5 and also prime.

Wait a minute.
...

So, the only number that satisfies both conditions is 5. Therefore, the function should
return True only when the input is 5, and False otherwise.

So the steps for the function would be:

...

But wait, what if someone passes a non-integer, like a float? But the problem says
"number", which could be a float. But primes are defined for integers. So perhaps the
function should first check if the input is an integer. Because 5.0 is equal to 5, but
it's a float. So maybe we need to check if n is an integer and equals 5.

...

So, that's the efficient way. Because checking for primality for numbers other than 5 is
unnecessary. Because any other number divisible by 5 is either 5 (prime) or a multiple
of 5 (not prime). So, the function can be optimized to just check if the number is 5.

Therefore, the code is correct and efficient.
</think>

```python
def check_prime_and_divisible_by_five(n):
    return n == 5 and isinstance(n, int)
```
```

(NVIDIA) Llama-Nemotron: Efficient Reasoning Models. 2025 https://arxiv.org/pdf/2505.00949

The lab

Notebook 2: Supervised fine-tuning (SFT) for large language models

1. Evaluate the baseline Model Llama-3.2-3B-Instruct
2. Prepare the dataset for supervised fine-tuning
3. Initialize SFT Recipe
4. Launch the supervised fine-tunning
5. Evaluating tuned model - did we learn anything?

NVIDIA DEEP LEARNING INSTITUTE

2. Supervised Fine-Tuning (SFT) for Large Language Models

A language model that has only undergone pretraining on next-token prediction is often not very practical for interactive tasks. Interacting with such a model can feel like talking to a website — it produces syntactically correct text, but it lacks an understanding of how to carry on a coherent or helpful conversation.

Supervised Fine-Tuning (SFT) addresses this limitation by training the model on a carefully constructed dataset of input-output examples. These examples typically represent desirable behaviors, such as answering questions, following instructions, or maintaining a specific tone. By training on such dataset in a supervised way, the model learns to align its responses with human expectations, making it significantly more useful for real-world applications.

Moreover, supervised fine-tuning can introduce new capabilities to a model, which has already been chat-aligned.

Motivation behind Supervised Fine-Tuning

In practice Supervised Fine-Tunning allows to achieve the following goals:

1. **Language adaptation**: Higher fluency and cultural knowledge in a target language.
2. **Domain expertise**: Working with accurate terminology and improving performance within specialized fields (law, medicine, finance, ...)
3. **Stylistic control**: Customizing model tone and conversation style (formal, friendly, technical, ...)
4. **Reasoning**: In recent reasoning models, SFT is used to introduce reasoning patterns before or instead of conducting Reinforcement Learning training to improve the model performance.

In this notebook, we will explore all four objectives focusing on improving a model's ability to solve mathematical problems in Spanish, demonstrating how fine-tuning can improve performance on specialized tasks in non-English languages.

This notebook is organized as follows:

1. Dataset for Supervised Fine-Tuning
2. Evaluate the baseline Model `Llama-3.2-3B-Instruct`
3. Prepare the Dataset for Supervised Fine-Tuning
4. Initialize SFT Recipe
5. Launch the Supervised Fine-Tunning
6. Evaluate the SFTed model - did we learn anything?

Resuelve el siguiente problema matemático. Asegúrate de poner la respuesta (y solo la respuesta) dentro de \boxed{}.

Si $f(x) = \frac{x+1}{3x-4}$, ¿cuál es el valor de $f(5)$?

```
Running inference...
Question: Resuelve el siguiente problema matemático. Asegúrate de poner la respuesta (y solo la respuesta) dentro de \boxed{}.

Si $f(x)=\dfrac{x+1}{3x-4}$, ¿cuál es el valor de $f(5)$?
Answer: \en caja{\dfrac{9}{11}}

--------------------------------------------------------------
```

```
Running inference...
Question: Resuelve el siguiente problema matemático. Asegúrate de poner la respuesta (y solo la respuesta) dentro de \boxed{}.

Si $f(x)=\dfrac{x+1}{3x-4}$, ¿cuál es el valor de $f(5)$?
Answer: Estoy bien, así que necesito encontrar el valor de f(5) para la función f(x) = (x + 1)/(3x - 4). Déjame pensar. Hmm, cuando se da una función, como f(x) = (x + 1)/(3x -
4), para encontrar f(5), simplemente sustituyo x con 5 en la función. Déjame escribir eso.

Así, f(5) sería (5 + 1)/(3*5 - 4). Déjame calcular el numerador primero. 5 más 1 es 6. Luego el denominador es 3 veces 5, que es 15, menos 4. Así, 15 - 4 es 11. Así, el valor es 6
dividido por 11. Déjame verificar eso de nuevo. 15 menos 4 es definitivamente 11. Así, 6 dividido por 11. Déjame calcular 6 dividido por 11. 6 dividido por 11 es aproximadamente
0.5454, pero como la pregunta no especifica si debe ser exacto o aproximado, debería dejarlo como 6/11. Así, f(5) es 6/11. Déjame verificar si hay un denominador común para simpli
ficarlo. El numerador es 6 y el denominador es 11. No hay factores comunes excepto 1, así que 6/11 ya está en su forma más simple. Así, la respuesta debería ser 6/11.

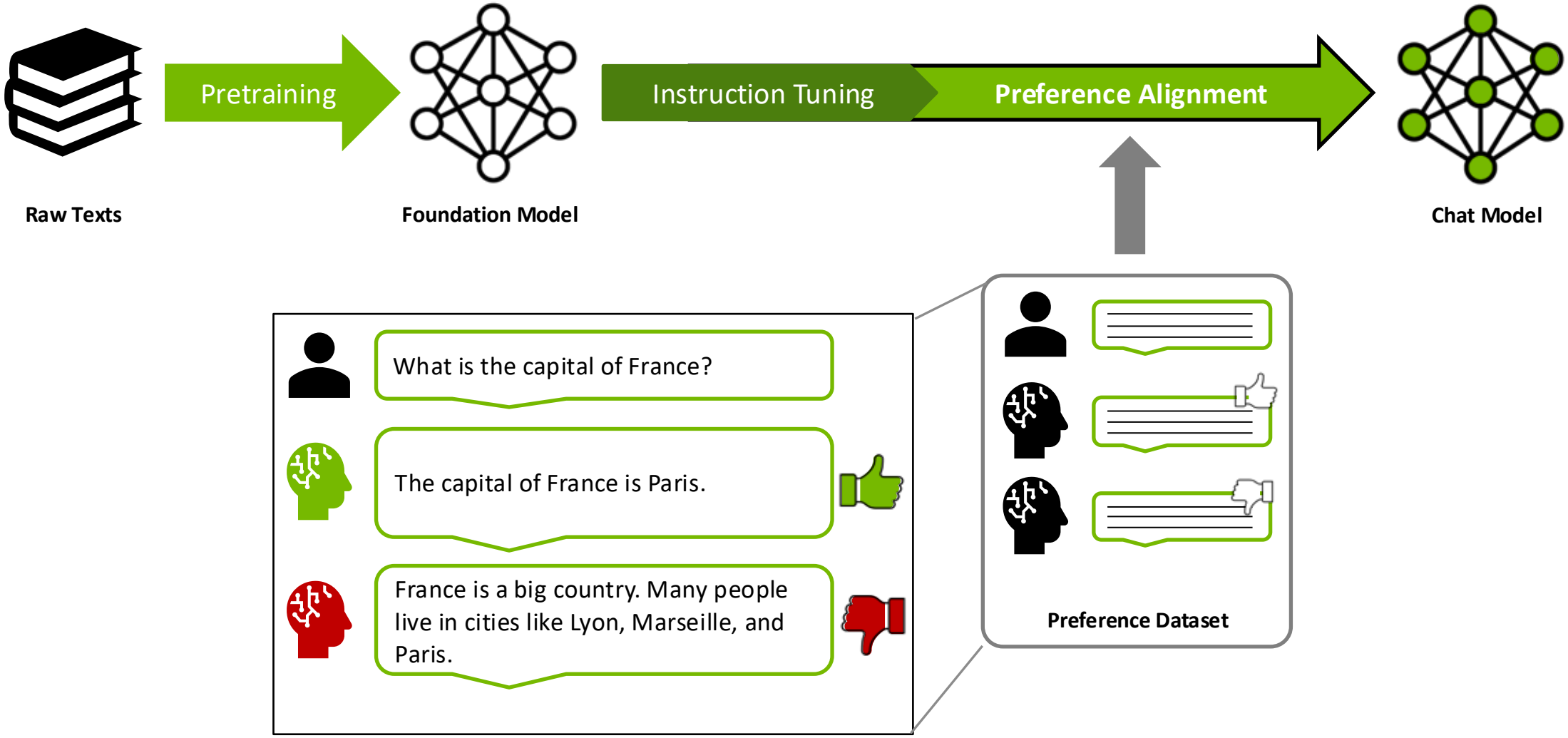
\boxed{\dfrac{6}{11}}
```

Agenda

- LLM Training Recap
- Continued Pretraining - CPT
- Supervised Fine-Tuning - SFT
- **Preference Alignment and RL**
- Summary

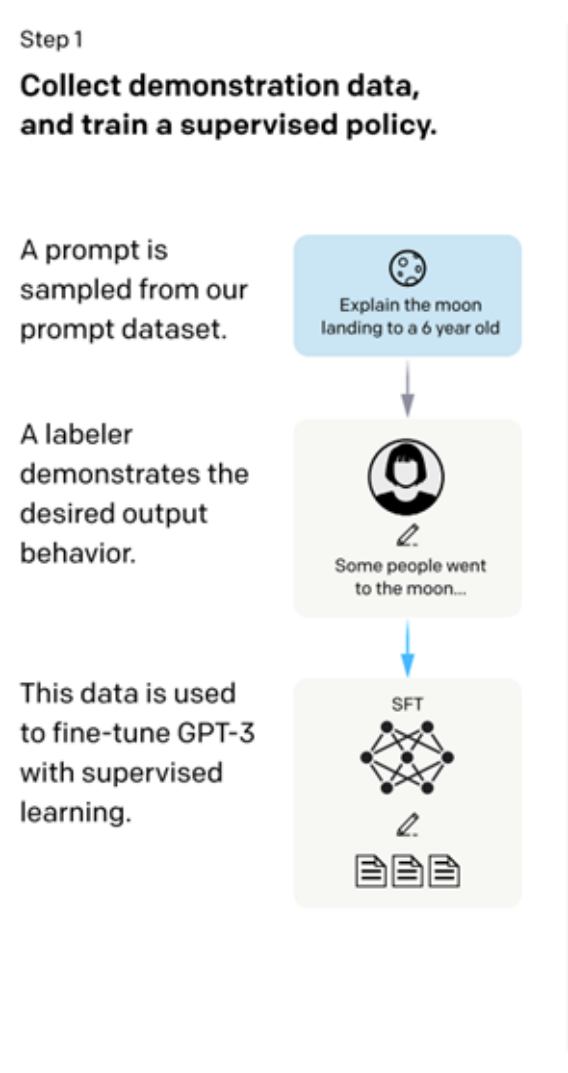
Preference alignment follows supervised fine-tuning

Training on human preference data for alignment – adjusting to human feedback



Reinforcement learning from human feedback

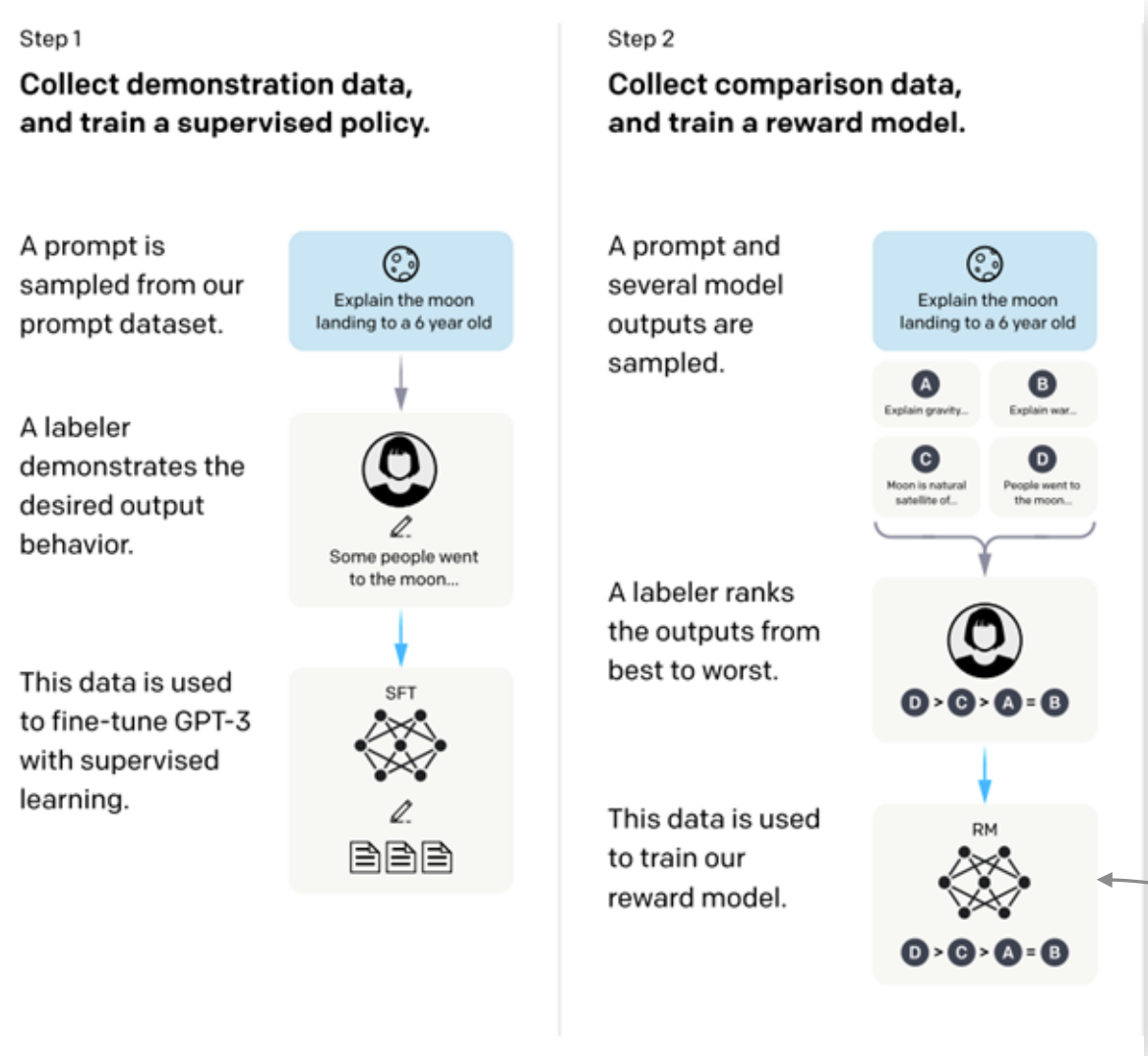
RLHF starts from fine-tuning LLM on manually annotated samples



Ouyang, Long, et al. "Training language models to follow instructions with human feedback." *Advances in neural information processing systems* 35 (2022): 27730-27744.

Reinforcement learning from human feedback

Ranked outputs are used to train the reward model (RM)

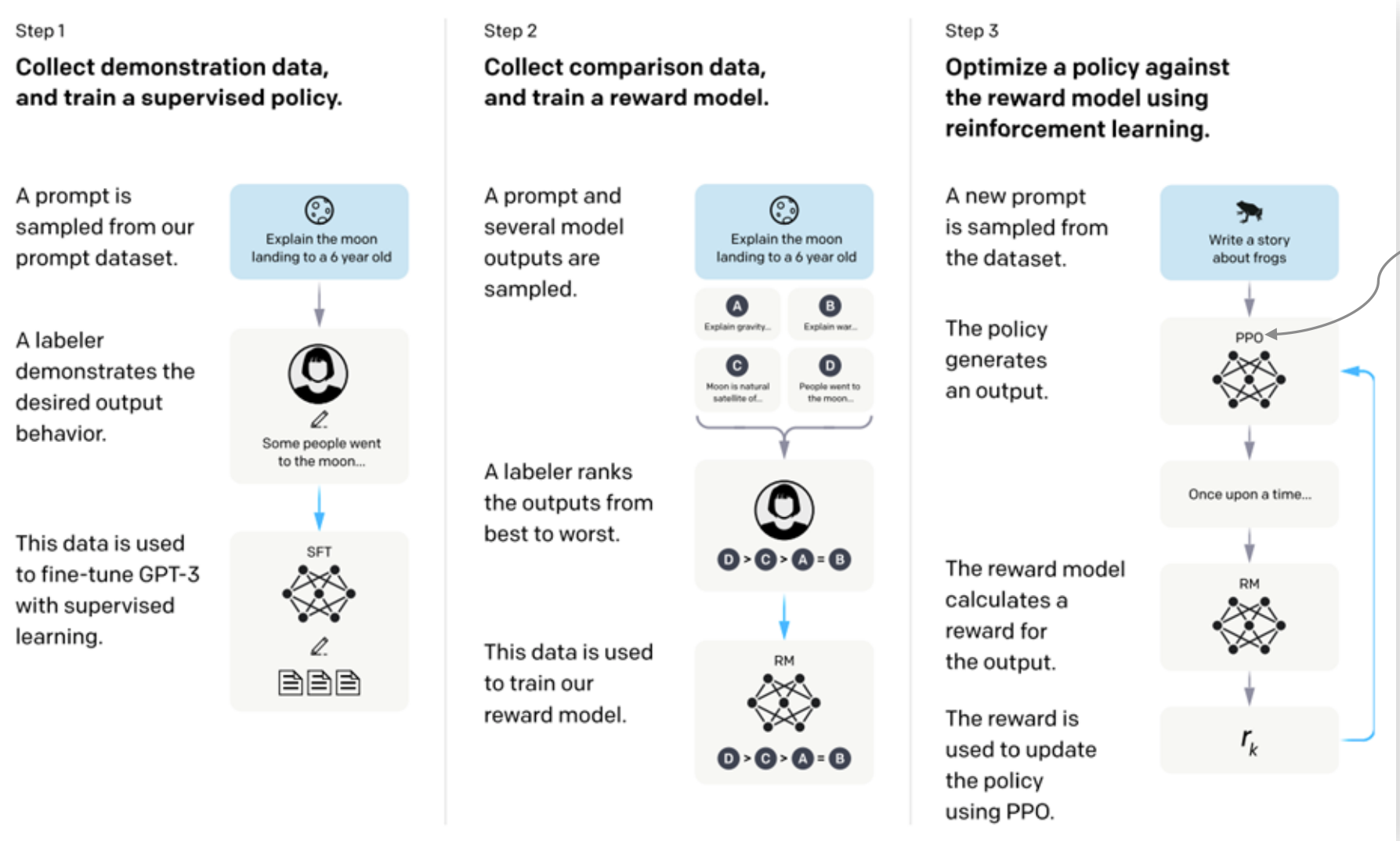


6B parameters in the original paper

Ouyang, Long, et al. "Training language models to follow instructions with human feedback." *Advances in neural information processing systems* 35 (2022): 27730-27744.

Reinforcement learning from human feedback

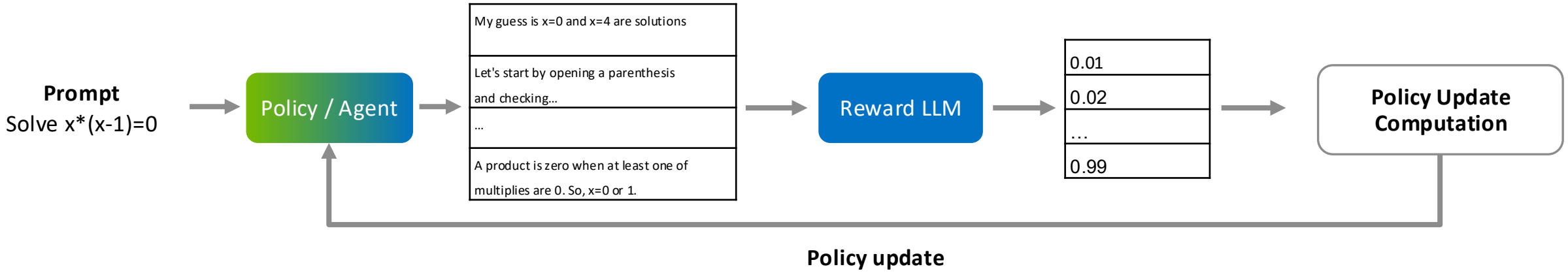
Reward model is used in proximal policy optimization



Ouyang, Long, et al. "Training language models to follow instructions with human feedback." *Advances in neural information processing systems* 35 (2022): 27730-27744.

Proximal policy optimization

Reinforcement learning in LLM world



Simplified RL objective: $\max_{\pi_\theta} \mathbb{E}_{x\sim\mathcal{D}, y\sim\pi_\theta(y|x)}\left[r_\phi(x,y)\right]$

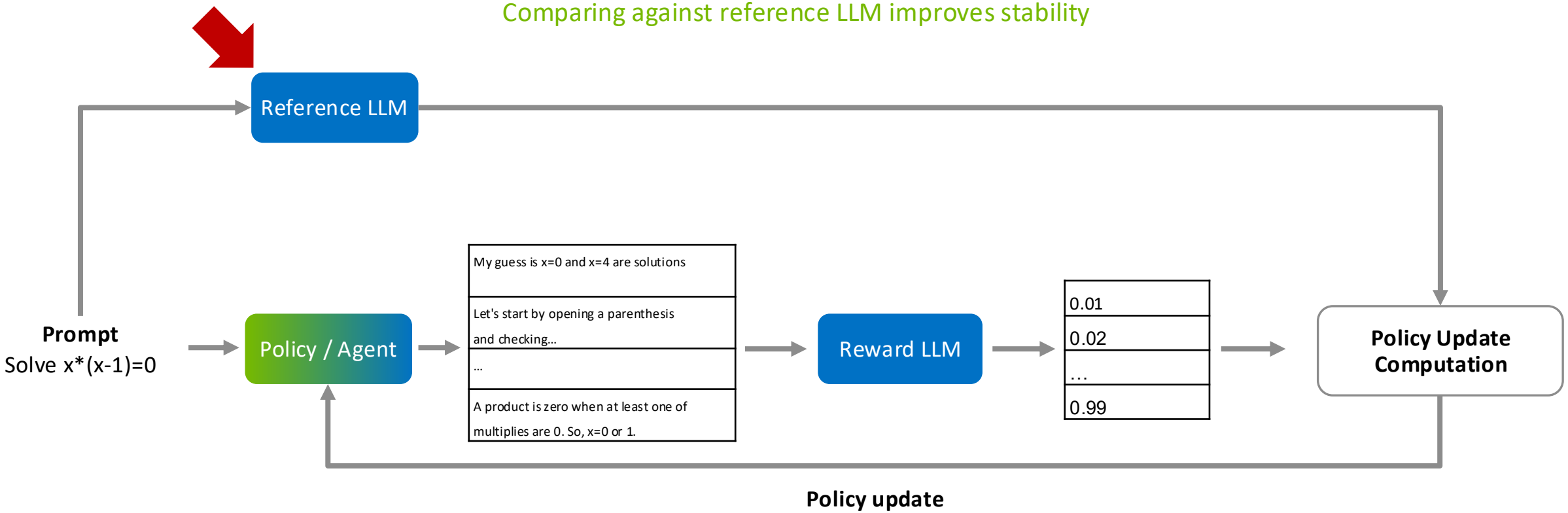
Inference Only

Training & Inference

Ouyang, Long, et al. "Training language models to follow instructions with human feedback." *Advances in neural information processing systems* 35 (2022): 27730-27744.

Proximal policy optimization

Comparing against reference LLM improves stability



Simplified RL objective:

$$\max_{\pi_\theta} \mathbb{E}_{x\sim\mathcal{D}, y\sim\pi_\theta(y|x)}\left[r_\phi(x,y)\right] - \beta \mathbb{D}_{\mathrm{KL}}\left[\pi_\theta(y \mid x) \,||\, \pi_{\mathrm{ref}}(y \mid x)\right]$$

KL term penalizes divergence from reference model

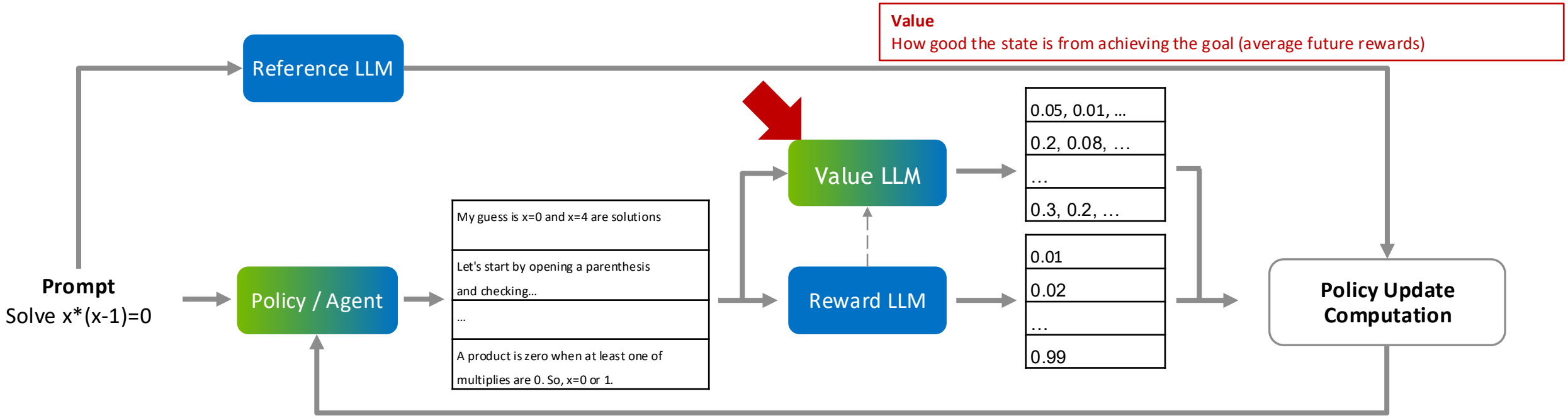
Inference Only

Training & Inference

Ouyang, Long, et al. "Training language models to follow instructions with human feedback." *Advances in neural information processing systems* 35 (2022): 27730-27744.

Proximal policy optimization

Advantage and value



Policy update

Simplified Full loss: $$L_t(\theta) = r_t(\theta)\hat{A}_t - \beta\,\mathrm{KL}[\pi_{\theta_{\text{old}}}, \pi_\theta]$$

Advantage
How much good/bad an action compared to the average actions

$$\hat{A}_t = -V(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V(s_T)$$

Inference Only

Training & Inference

Ouyang, Long, et al. "Training language models to follow instructions with human feedback." *Advances in neural information processing systems* 35 (2022): 27730-27744.

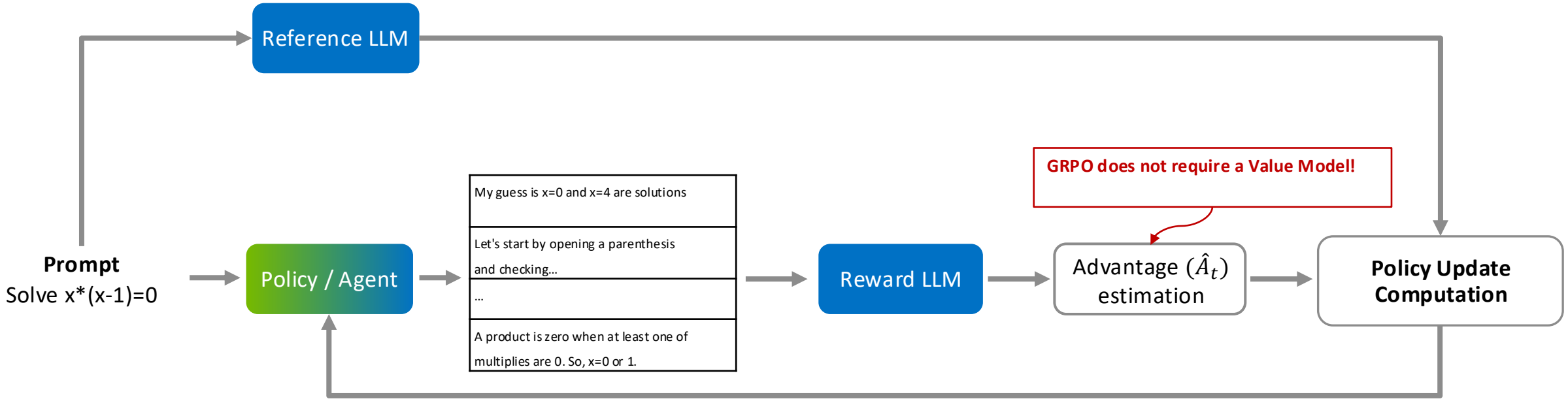
“

Do we need a value LLM?

”

Group relative policy optimization (GRPO)

GRPO estimates advantage values from rewards



Advantage
Estimated from Reward Model

$$A_i = \frac{r_i - mean(\{r_1, r_2, \cdots, r_G\})}{std(\{r_1, r_2, \cdots, r_G\})}.$$

Inference Only

Training & Inference

Shao, Zhihong, et al. "Deepseekmath: Pushing the limits of mathematical reasoning in open language models." arXiv preprint arXiv:2402.03300 (2024).

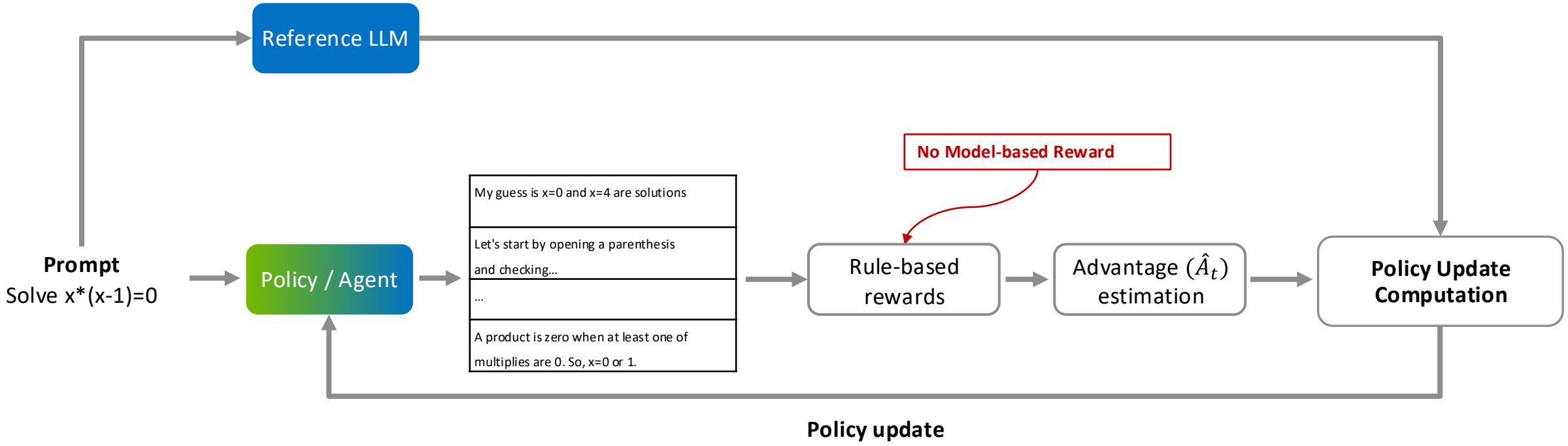
“

Do we need a reward LLM?

”

GRPO – DeepSeek R1 approach to alignment

Rule-based verifiable rewards



Inference Only

Training & Inference

Rewards are computed based on rules

- Being correct and following format = higher reward
- Only works for verifiable problems

(DeepSeek-AI) DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. 2025. https://arxiv.org/abs/2501.12948

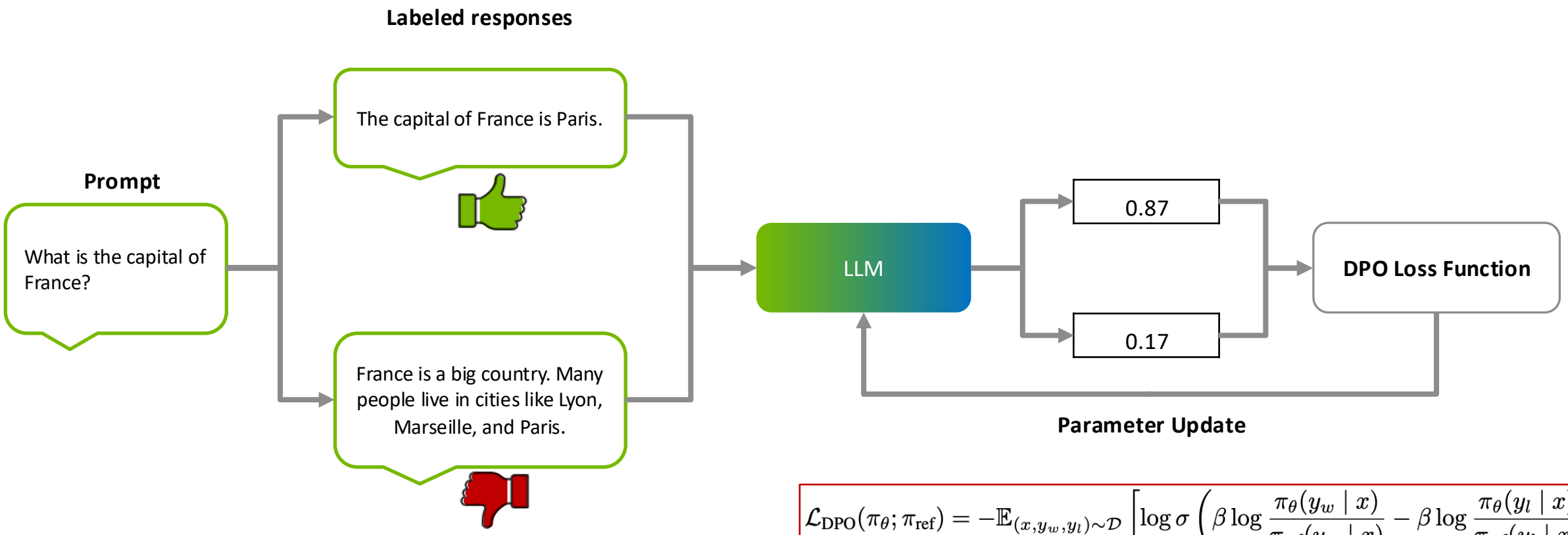
“

Do we need reinforcement learning?

”

Direct preference optimization

Clever technique to train directly on preference data



$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta\log\frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]$$

- No reinforcement learning or additional models
- Can only train on labeled preference data

Inference Only

Training & Inference

(Stanford University) Direct Preference Optimization: Your Language Model is Secretly a Reward Model. 2023. https://arxiv.org/abs/2305.18290

NeMo-RL

Scalable and efficient post-training library designed for models of all sizes ranging from 1 GPU to 1000s

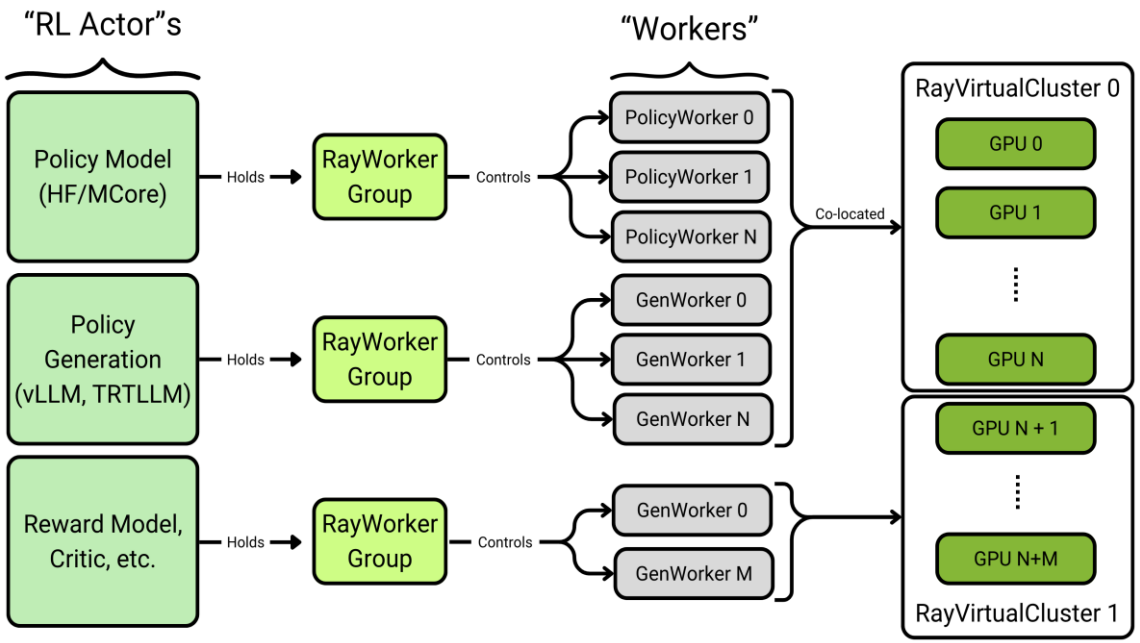
- **Who it's for**

For enterprise customers to do efficient post-training with a pre-trained foundational model

- **What's its value / differentiator**
  - **Scalability**: high-performance implementation with Megatron Core, supporting parallelism techniques for large models and contexts, resource management using Ray for different hardware configurations
  - **End-to-End**: Supports full RLHF pipeline from SFT to reward model training to GRPO
  - **Flexibility:** Open source with commercial usable license
  - **Usability:** Hugging Face and vLLM integration with detailed documentation and practical examples

- **How to use it?**
  - Github open source
  - In the future exposed as a microservice

Co-ordinates and resources RL Actors

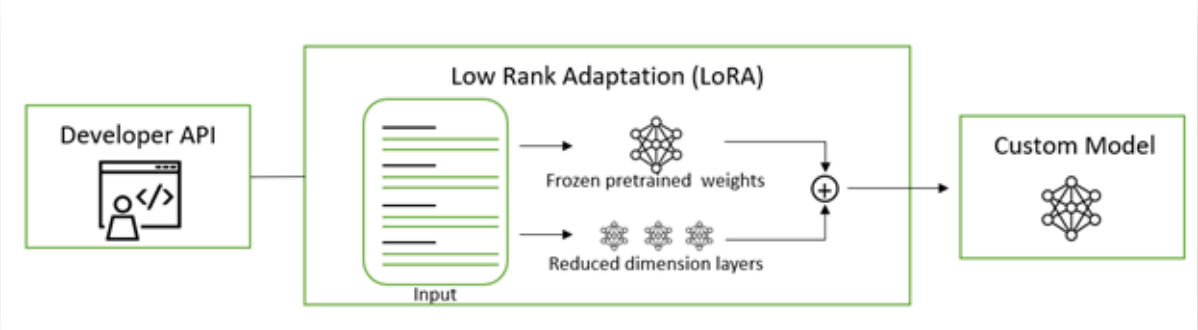


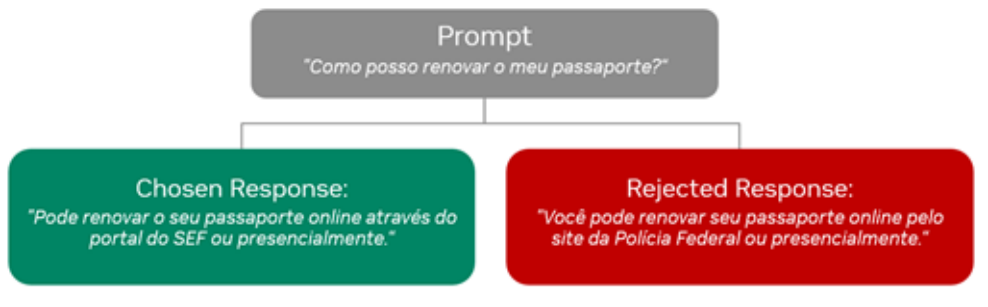
NVIDIA/NeMo-RL: Scalable toolkit for efficient model reinforcement

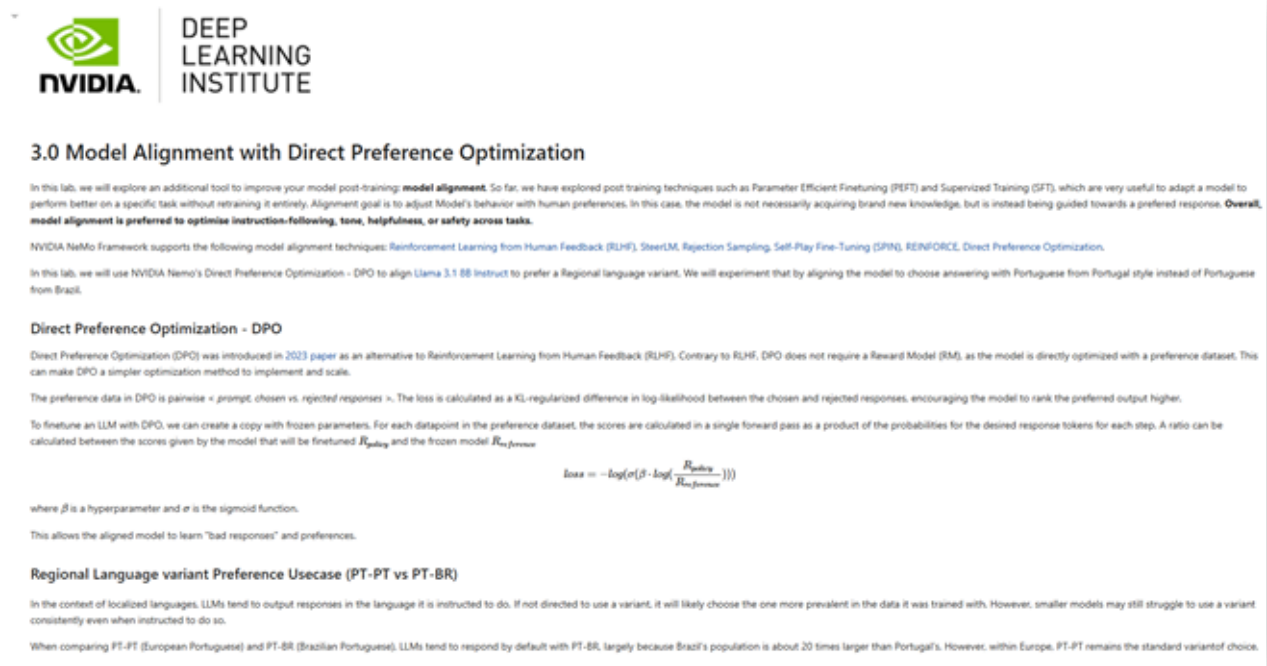
The lab

Notebook 3: Model alignment with direct preference optimization

1. Prepare the dataset for DPO Alignment
2. Evaluate the Llama-3.1-8B-Instruct performance
3. Align Llama-3.1-8B-Instruct with DPO
4. Evaluated the aligned model







NVIDIA DEEP LEARNING INSTITUTE

3.0 Model Alignment with Direct Preference Optimization

In this lab, we will explore an additional tool to improve your model post-training: **model alignment**. So far, we have explored post training techniques such as Parameter Efficient Finetuning (PEFT) and Supervized Training (SFT), which are very useful to adapt a model to perform better on a specific task without retraining it entirely. Alignment goal is to adjust Model's behavior with human preferences. In this case, the model is not necessarily acquiring brand new knowledge, but is instead being guided towards a prefered response. **Overall, model alignment is preferred to optimise instruction-following, tone, helpfulness, or safety across tasks.**

NVIDIA NeMo Framework supports the following model alignment techniques: Reinforcement Learning from Human Feedback (RLHF), SteerLM, Rejection Sampling, Self-Play Fine-Tuning (SPIN), REINFORCE, Direct Preference Optimization.

In this lab, we will use NVIDIA Nemo's Direct Preference Optimization - DPO to align Llama 3.1 8B Instruct to prefer a Regional language variant. We will experiment that by aligning the model to choose answering with Portuguese from Portugal style instead of Portuguese from Brazil.

Direct Preference Optimization - DPO

Direct Preference Optimization (DPO) was introduced in 2023 paper as an alternative to Reinforcement Learning from Human Feedback (RLHF). Contrary to RLHF, DPO does not require a Reward Model (RM), as the model is directly optimized with a preference dataset. This can make DPO a simpler optimization method to implement and scale.

The preference data in DPO is pairwise *< prompt, chosen vs. rejected responses >*. The loss is calculated as a KL-regularized difference in log-likelihood between the chosen and rejected responses, encouraging the model to rank the preferred output higher.

To finetune an LLM with DPO, we can create a copy with frozen parameters. For each datapoint in the preference dataset, the scores are calculated in a single forward pass as a product of the probabilities for the desired response tokens for each step. A ratio can be calculated between the scores given by the model that will be finetuned R_{policy} and the frozen model $R_{reference}$

$$loss = -log(\sigma(\beta \cdot log(\frac{R_{policy}}{R_{reference}})))$$

where β is a hyperparameter and σ is the sigmoid function.

This allows the aligned model to learn "bad responses" and preferences.

Regional Language variant Preference Usecase (PT-PT vs PT-BR)

In the context of localized languages, LLMs tend to output responses in the language it is instructed to do. If not directed to use a variant, it will likely choose the one more prevalent in the data it was trained with. However, smaller models may still struggle to use a variant consistently even when instructed to do so.

When comparing PT-PT (European Portuguese) and PT-BR (Brazilian Portuguese), LLMs tend to respond by default with PT-BR, largely because Brazil's population is about 20 times larger than Portugal's. However, within Europe, PT-PT remains the standard variantof choice.

Agenda

- LLM Training Recap
- Continued Pretraining - CPT
- Supervised Finetuning - SFT
- Reinforcement Learning - RL
- **Summary**

When to use what?

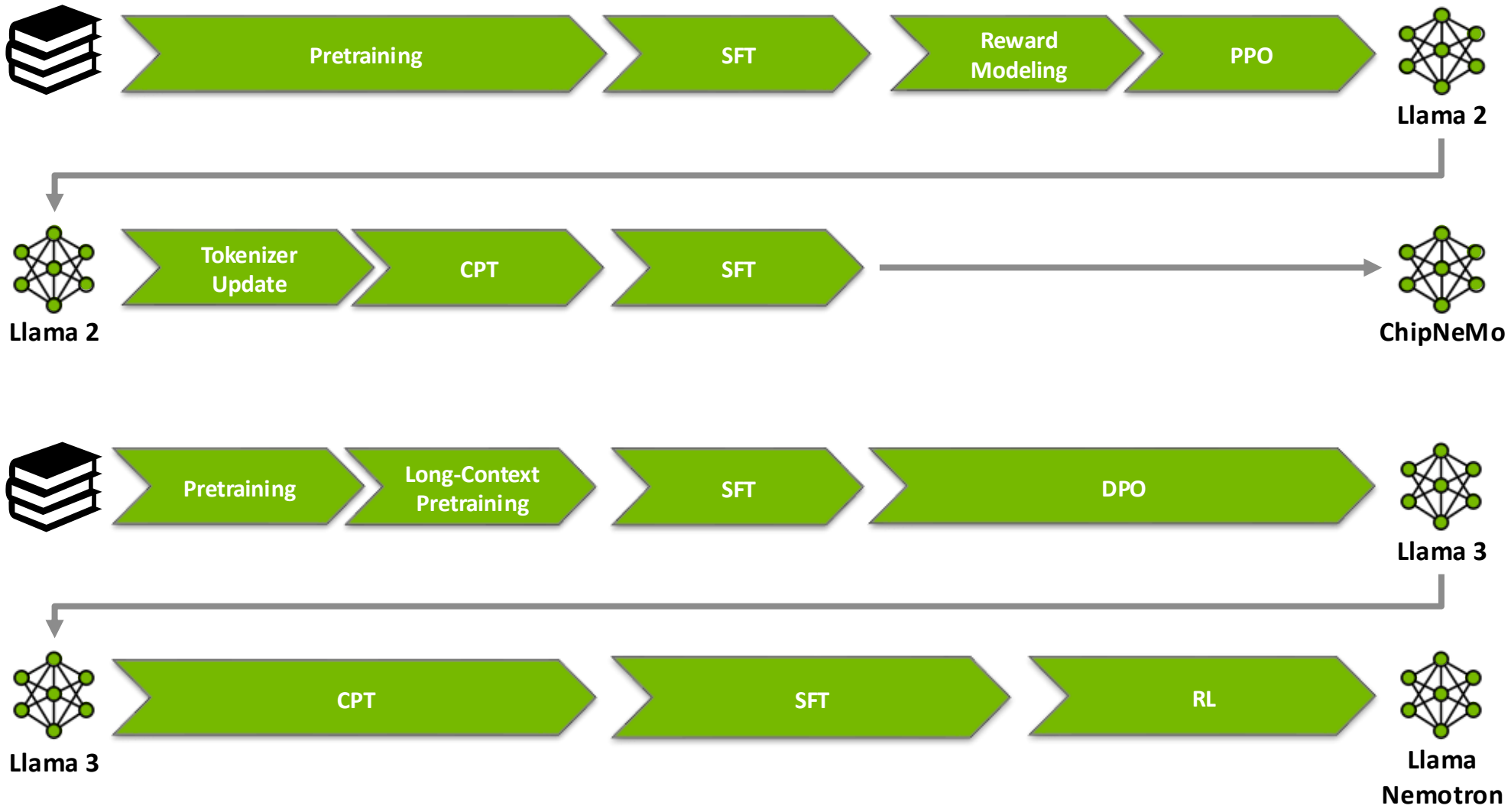
Different post training techniques have different purposes

	Continued Pretraining CPT	Supervised Fine-Tuning SFT	Preference Alignment PPO, GRPO, DPO	
Purpose	• Language understanding • Factual knowledge	• Instruction Following, Chat • Reasoning Skills	• Aligning to human preferences	
Data	• Raw texts	• Prompt + answers	• Prompt + answers + ranking	
Pros	• Easy to collect data, scales well	• Simple and stable to train	• Fixes behavior not covered by SFT • Easier to collect data compared to SFT	
Caveats	• Need to re-run alignment • Determining data blend • Training stability	• Hard to collect good quality annotated data	PPO	• Need for additional models • Sensitive training process • Computationally expensive
			GRPO	• Computationally more efficient than PPO
			DPO	• No reward model • Directly targets human preferences • Faster alignment

- All post-training techniques can be used with PEFT
- For most existing latest foundation LLMs, RAG is an easier alternative for Factual Knowledge

Recap – how LLMs are trained and tuned?

Different techniques to train and customize LLMs



Thank you!

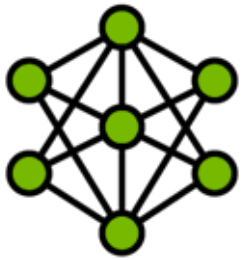
Bonus: Tokenizer Adaptation

Reminder: LLMs do not work directly with texts

Most domains have highly specific terminology

Aminoglycoside, erythromycin, secukinumab, norfluoxetine, enalapril, …

?



LLM

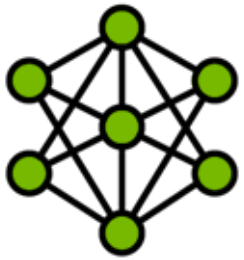
Why tokenizer choice matters for specific domains

More tokens = more compute, lower performance

Aminoglycoside, erythromycin, secukinumab, norfluoxetine, enalapril, …

Tokenization

Aminoglycoside, erythromycin, secukinumab, norfluoxetine, enalapril, …



LLM

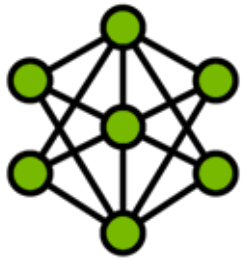
Why tokenizer choice matters for specific domains

More tokens = more compute, lower performance

Aminoglycoside, erythromycin, secukinumab, norfluoxetine, enalapril, …

Tokenization

Aminoglycoside, erythromycin, secukinumab, norfluoxetine, enalapril, …



LLM

Inefficient tokenization:

- Increases sequence length, training and inference costs
- Reduces semantic clarity
- Distorts statistical patterns
- Hurts downstream task performance

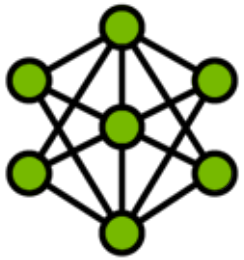
Tokenizer adaptation – optional step of continued pretraining

Ideally 1 domain term = 1 token

Aminoglycoside, erythromycin, secukinumab, norfluoxetine, enalapril, …

Domain-adapted tokenization

Aminoglycoside, erythromycin, secukinumab, norfluoxetine, enalapril, …

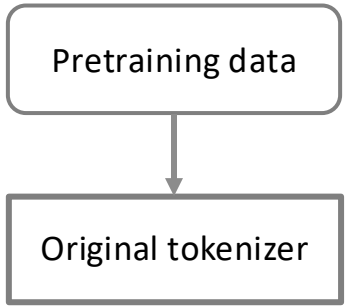


LLM

Tokenizer adaptation – ChipNeMo approach

1. Train a new tokenizer on domain data

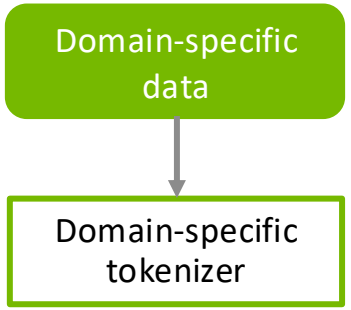
Original tokenizer



the, a, but, cat, dog, …

Tokenizer training

Domain-specific tokenizer

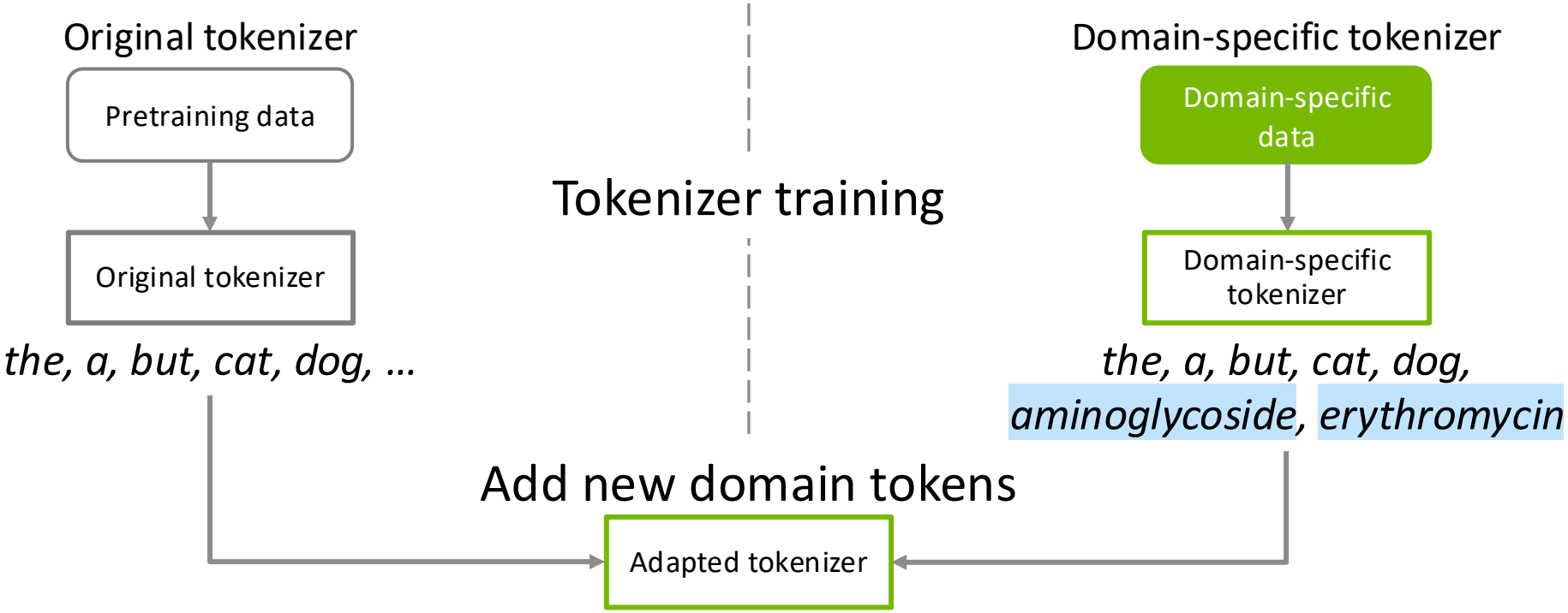


the, a, but, cat, dog, aminoglycoside, erythromycin

ChipNeMo: Domain-Adapted LLMs for Chip Design. Mingjie Liu et al. arXiv:2311.00176.

Tokenizer adaptation – ChipNeMo approach

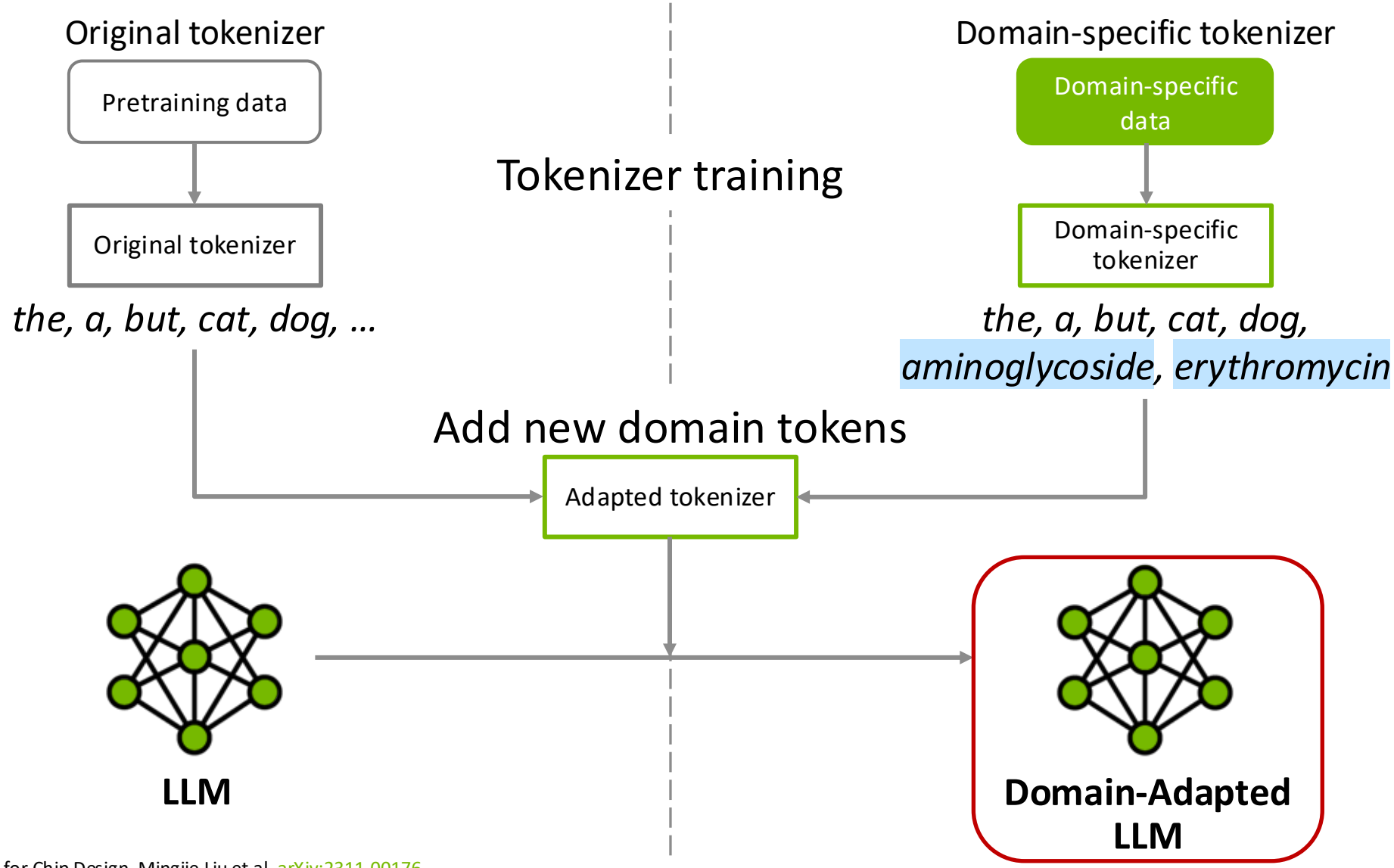
2. Update original tokenizer with domain-specific tokens



ChipNeMo: Domain-Adapted LLMs for Chip Design. Mingjie Liu et al. arXiv:2311.00176.

Tokenizer adaptation – ChipNeMo approach

3. Update LLM to understand new tokens



ChipNeMo: Domain-Adapted LLMs for Chip Design. Mingjie Liu et al. arXiv:2311.00176.